CHAPTER 1

UNDERSTAND FUNDAMENTAL PRINCIPLES OF ALGORITHMS

Definition of an Algorithm

Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output.

An algorithm can also be defined as a step-by-step procedure of solving a mathematical problem.

Algorithms are generally created independent of underlying languages i.e. an algorithm can be implemented in more than one programming language

The following are categories of algorithms from the data structure point of view.

- **Search Algorithm –** used to search an item in a data structure.
- **Sort Algorithm –** used to sort items in a certain order.
- **Insert Algorithm –** used to insert item in a data structure
- **Update Algorithm –** used to update an existing Item in a data structure.
- **Delete Algorithm –** to delete an existing item from a data structure

Characteristics of an Algorithm

Not all procedures qualify to be called an algorithm. An algorithm should have the following characteristics:

- **Unambiguous –** it should be clear free from ambiguity. Each step and their inputs/outputs should be clear and must lead to one meaning.
- **Input –** Must have 0 or more well defined inputs.
- **Outputs –** should have 1 or more well defined outputs, and should match the desired outputs.
- **Finiteness –** It must terminate after a finite number of steps.
- **Feasibility -** should be feasible with the available resources
- **Independent –** Should have step-by-step direction, which should be independent of any programming code.

Principles of algorithm writing

There are no well-defined standards for writing algorithms. Rather, it is problem and resource dependent.

Algorithms are never written to support a particular programming code.

As we know that all programming languages share basic code constructs like control structures (loops & flow-control). These common constructs can be used to write an algorithm.

Algorithm writing is a process and is executed after the problem domain is well defined. That is, we should know the problem domain, for which we are designing a solution.

Example

Problem − Design an algorithm to add two numbers and display the result.

Step 1 − START

Step 2 − declare three integers a, b & c

Step 3 − define values of a & b

Step 4 − add values of a & b

Step 5 − store output of step 4 to c

Step 6 − print c

Step 7 − STOP

Algorithms tell the programmers how to code the program. Alternatively, the algorithm can be written as −

Step 1 − START ADD

Step 2 − get values of a & b

Step 3 − c ← a + b

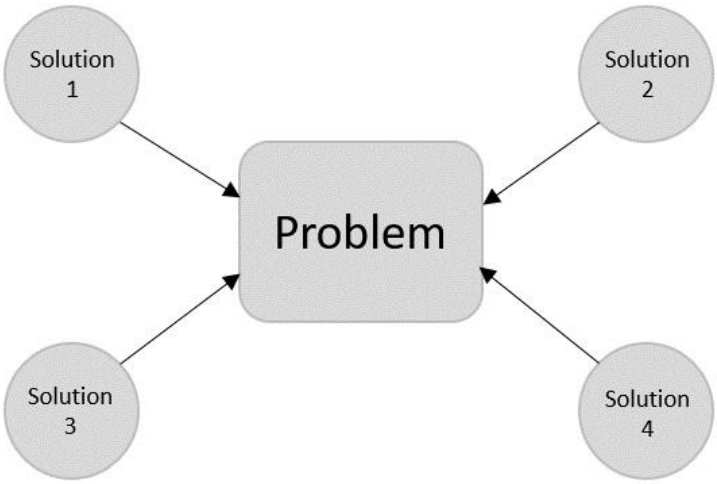
Step 4 − display c

Step 5 − STOP

In design and analysis of algorithms, usually the second method is used to describe an algorithm. It makes it easy for the analyst to analyze the algorithm ignoring all unwanted definitions. He can observe what operations are being used and how the process is flowing.

Writing step – numbers is optional.

We design an algorithm to get a solution of a given problem. A problem can be solved in more than one ways.



Hence, many solution algorithms can be derived for a given problem. The next step is to analyze those proposed solution algorithms and implement the best suitable solution.

Algorithm Analysis

Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation. They are the following −

A Priori Analysis − this is a theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation.

A Posterior Analysis − this is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required are collected.

Algorithm analysis deals with the execution or running time of various operations involved. The **running time** of an operation can be defined as the number of computer instructions executed per operation.

Complexities of algorithms

Suppose **X** is an algorithm and **n** is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the efficiency of X.

Time Factor − Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.

Space Factor − Space is measured by counting the maximum memory space required by the algorithm.

The complexity of an algorithm **f(n)** gives the running time and/or the storage space required by the algorithm in terms of **n** as the size of input data.

Space Complexity

Space complexity is defined as the process of determine a formula for prediction of how much memory space will be required for the successful execution of the algorithm. The memory space we generally consider is the space of primary memory.

Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components −

A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.

A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.

Space complexity S(P) of any algorithm P is S(P) = C + SP(I), where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I. Following is a simple example that tries to explain the concept −

Algorithm: SUM(A, B)

Step 1 − START

Step 2 − C ← A + B + 10

Step 3 − Stop

Here we have three variables A, B, and C and one constant. Hence S(P) = 1 + 3. Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Time Complexity

Time complexity is defined as the process of determine a formula for total time required towards execution of that algorithm. This calculation will be independent of implementation details and programming language.

Time complexity of an algorithm represents the amount of time required by the algorithm to run to completion. Time requirements can be defined as a numerical function T(n), where T(n) can be measured as the number of steps, provided each step consumes constant time.

Asymptotic notations – They are used to make meaningful statements about efficiency of algorithms. These notations helps us to make approx. but meaningful assumptions about the time and space complexity.

Commonly used asymptotic notations are:

- Big-oh Notation (o) (upper bound) maximum time complexity an algorithm will take (n)
- Big – Omegah (Ω) (Lower bound) minimum time complexity an algorithm will take (1)
- Big – Theta (Θ) (Tight bound) in between the upper and lower limit

For example, addition of two n-bit integers takes **n** steps. Consequently, the total computational time is $T(n) = c * n$, where c is the time taken for the addition of two bits. Here, we observe that T(n) grows linearly as the input size increases.

Greedy algorithms

Greedy is an algorithmic paradigm that builds up a solution **piece by piece**, always choosing the next piece that offers the most **obvious** and **immediate benefit**. Greedy algorithms are used for optimization problems.

An optimization problem can be solved using **Greedy** if the problem has the following property:

Greedy Choice Property: The optimal solution can be constructed by making the **best local choice** at each step.

Optimal Substructure: The optimal solution to the problem contains the optimal solutions to its sub-problems.

Characteristics of Greedy Algorithm

Greedy algorithms are **simple and easy** to implement.

They are efficient in terms of **time complexity,** often providing quick solutions.

These algorithms do not reconsider previous choices, as they make decisions based on current information without looking ahead.

How Greedy Algorithm Works

Start with the **initial state** of the problem. This is the **starting point** from where you begin making choices.

Evaluate **all possible choice**s you can make from the current state. Consider all the options available at that specific moment.

Choose the option that seems best at that moment, regardless of future consequences. This is the **"greedy"** part - **you take the best option available now, even if it might not be the best in the long run.**

Move to the new state based on your chosen option. This becomes your **new starting point** for the next iteration.

Repeat **steps 2-4** until you reach the **goal state** or no further progress is possible. Keep making the best local choices until you reach the end of the problem or get stuck.

Example:

Let's say you have a **set of coins** with values **[1, 2, 5, 10]** and you need to give **minimum number of coin** to someone change for **39**.

*The **greedy algorithm** for making change would work as follows:*

Step-1: *Start with the **largest coin** value that is **less than or equal** to the amount to be changed. In this case, the largest coin less than or equal to **39** is 10.*

Step- 2: *Subtract the largest coin value from the amount to be changed, and **add** the coin to the solution. In this case, **subtracting 10 from 39 gives 29**, and we add **one 10-coin** to the solution.*

*Repeat **steps 1 and 2** until the amount to be changed becomes 0.*

The **greedy algorithm is not always the optimal solution** for every optimization problem, as shown in the example below.

When using the greedy approach to make change for the **amount 20** with the **coin denominations [18, 1, 10**], the algorithm starts by selecting the **largest coin** value that is **less than or equal** to the target amount. In this case, the **largest coin is 18,** so the algorithm selects **one 18 coin**. After **subtracting 18 from 20**, the **remaining amount is 2.**

At this point, the greedy algorithm chooses the **next largest coin less than or equal to 2,** which is 1. It then selects **two 1 coins** to make up the remaining amount. So, the greedy approach results in using **one 18 coin and two 1 coins.**

However, the greedy approach fails to find the optimal solution in this case. Although it uses **three coins**, a better solution would have been to use **two 10 coins**, resulting in a total of only two coins **(10 + 10 = 20)**

Divide and conquer algorithms

Divide and Conquer Algorithm is a problem-solving technique used to solve problems by dividing the main problem into sub-problems, solving them individually and then merging them to find solution to the original problem.

Working of Divide and Conquer Algorithm

Divide and Conquer Algorithm can be divided into three steps: **Divide**, **Conquer** and **Merge**.

The above diagram shows working with the example of Merge Sort, which is used for sorting

1. Divide:

Break down the original problem into smaller sub-problems.

Each sub-problem should represent a part of the overall problem.

The goal is to divide the problem until no further division is possible.

2. Conquer:

Solve each of the smaller sub-problems individually.

If a sub-problem is small enough, we solve it directly without further recursion.

The goal is to find solutions for these sub-problems independently.

3. Merge:

Combine the sub-problems to get the final solution of the whole problem.

Once the smaller sub-problems are solved, we recursively combine their solutions to get the solution of larger problem.

The goal is to formulate a solution for the original problem by merging the results from the sub-problems.

Characteristics of Divide and Conquer Algorithm

Dividing the Problem: The first step is to break the problem into smaller, more manageable -. This division can be done recursively until the sub-problems become simple enough to solve directly.

Independence of Sub-problems: Each sub-problem should be independent of the others, meaning that solving one sub-problem does not depend on the solution of another. This allows for parallel processing or concurrent execution of sub-problems, which can lead to efficiency gains.

Conquering Each Sub-problem: Once divided, the sub-problems are solved individually. This may involve applying the same divide and conquer approach recursively until the sub-problems become simple enough to solve directly, or it may involve applying a different algorithm or technique.

Combining Solutions: After solving the sub-problems, their solutions are combined to obtain the solution to the original problem. This combination step should be relatively efficient and straightforward, as the solutions to the sub-problems should be designed to fit together seamlessly.

Examples of Divide and Conquer Algorithm

1. Merge Sort:

We can use Divide and Conquer Algorithm to sort the array in ascending or descending order by dividing the array into smaller subarrays, sorting the smaller subarrays and then merging the sorted arrays to sort the original array.

2. Quicksort:

It is a sorting algorithm that picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.

Applications of Divide and Conquer Algorithm

The following are some standard algorithms that follow Divide and Conquer algorithm:

Binary Search is an efficient algorithm for finding an element in a sorted array by repeatedly dividing the search interval in half. It works by comparing the target value with the middle element and narrowing the search to either the left or right half, depending on the comparison.

Quicksort is a sorting algorithm that picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.

Merge Sort is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.

Closest Pair of Points The problem is to find the closest pair of points in a set of points in the x-y plane. The problem can be solved in O(n^2) time by calculating the distances of every pair of points and comparing the distances to find the minimum. The Divide and Conquer algorithm solves the problem in O(N log N) time.

Strassen's Algorithm is an efficient algorithm to multiply two matrices. A simple method to multiply two matrices needs 3 nested loops and is O(n^3). Strassen's algorithm multiplies two matrices in O(n^2.8974) time.

Cooley–Tukey Fast Fourier Transform (FFT) algorithm is the most common algorithm for FFT. It is a divide and conquer algorithm which works in O(N log N) time.

Karatsuba algorithm for fast multiplication does the multiplication of two binary strings in O(n1.59) where n is the length of binary string.

Advantages of Divide and Conquer Algorithm

Solving difficult problems: Divide and conquer technique is a tool for solving difficult problems conceptually. e.g. Tower of Hanoi puzzle. It requires a way of breaking the problem into sub-problems, and solving all of them as an individual cases and then combining sub- problems to the original problem.

Algorithm efficiency: The divide-and-conquer algorithm often helps in the discovery of efficient algorithms. It is the key to algorithms like Quick Sort and Merge Sort, and fast Fourier transforms.

Parallelism: Normally Divide and Conquer algorithms are used in multi-processor machines having shared-memory systems where the communication of data between processors does not need to be planned in advance, because distinct sub-problems can be executed on different processors.

Memory access: These algorithms naturally make an efficient use of memory caches. Since the sub-problems are small enough to be solved in cache without using the main memory that is slower one. Any algorithm that uses cache efficiently is called cache oblivious.

Disadvantages of Divide and Conquer Algorithm

Overhead: The process of dividing the problem into sub-problems and then combining the solutions can require additional time and resources. This overhead can be significant for problems that are already relatively small or that have a simple solution.

Complexity: Dividing a problem into smaller sub-problems can increase the complexity of the overall solution. This is particularly true when the sub-problems are interdependent and must be solved in a specific order.

Difficulty of implementation: Some problems are difficult to divide into smaller sub-problems or require a complex algorithm to do so. In these cases, it can be challenging to implement a divide and conquer solution.

Memory limitations: When working with large data sets, the memory requirements for storing the intermediate results of the subproblems can become a limiting factor.

CHAPTER 2

UNDERSTAND FUNDAMENTAL CONCEPTS OF DATA STRUCTURES

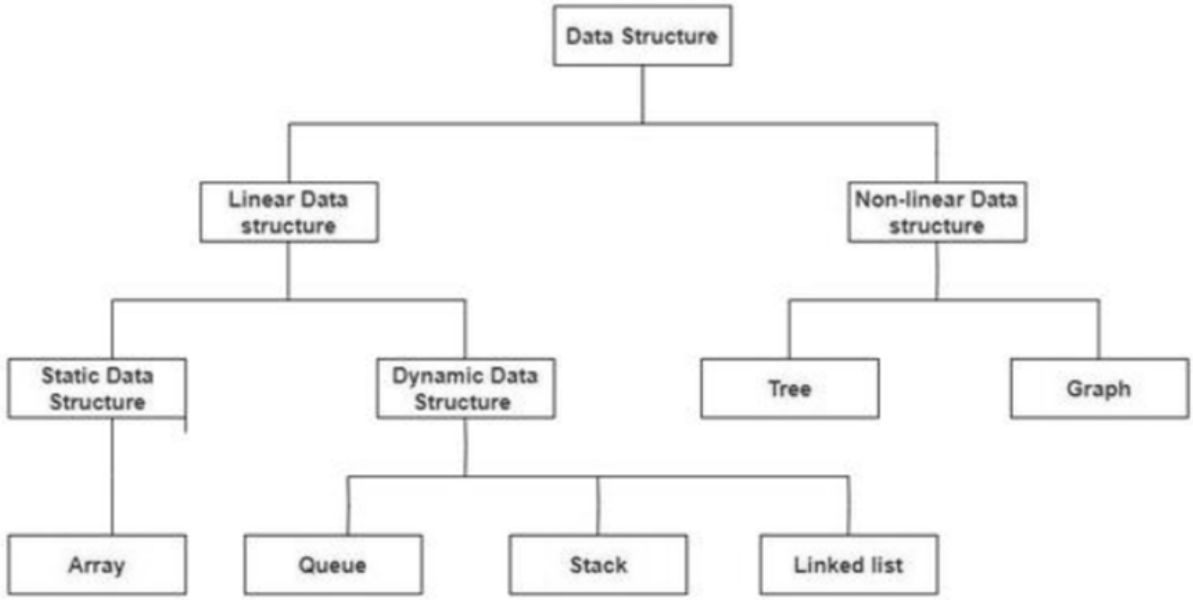
A **data structure** is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. It refers to the logical or mathematical representation of data, as well as the implementation in a computer program.

Classification:

Data structures can be classified into two broad categories:

Linear Data Structure: A data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements is called a linear data structure. Examples are array, stack, queue, etc.

Non-linear Data Structure: Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. Examples are trees and graphs.



Applications of Data Structures:

Data structures are used in a wide range of computer programs and applications, including:

Databases: Data structures are used to organize and store data in a database, allowing for efficient retrieval and manipulation.

Operating systems: Data structures are used in the design and implementation of operating systems to manage system resources, such as memory and files.

Computer graphics: Data structures are used to represent geometric shapes and other graphical elements in computer graphics applications.

Artificial intelligence: Data structures are used to represent knowledge and information in artificial intelligence systems.

Advantages of Data Structures:

The use of data structures provides several advantages, including:

Efficiency: Data structures allow for efficient storage and retrieval of data, which is important in applications where performance is critical.

Flexibility: Data structures provide a flexible way to organize and store data, allowing for easy modification and manipulation.

Reusability: Data structures can be used in multiple programs and applications, reducing the need for redundant code.

Maintainability: Well-designed data structures can make programs easier to understand, modify, and maintain over time.

Key concepts in data structures

Data Definition

Data Definition defines a particular data with the following characteristics.

Atomic − Definition should define a single concept.

Traceable − Definition should be able to be mapped to some data element.

Accurate − Definition should be unambiguous.

Clear and Concise − Definition should be understandable.

Data Object

Data Object represents an object having a data.

Data Type

Data type is a way to classify various types of data such as integer, string, etc. which determines the values that can be used with the corresponding type of data, the type of operations that can be performed on the corresponding type of data. There are two data types −

1. **Built-in Data Type**
2. **Derived Data Type**

Built-in Data Type (Primitive)

Those data types for which a language has built-in support are known as Built-in Data types. For example, most of the languages provide the following built-in data types.

- Integers
- Boolean (true, false)
- Floating (Decimal numbers)
- Character and Strings

Derived Data Type (Non-Premitive)

Those data types, which are implementation independent as they can be implemented in one or the other way, are known as derived data types. These data types are normally built by the combination of primary or built-in data types and associated operations on them. For example −

- List
- Array
- Stack
- Queue

Basic Operations

The data in the data structures are processed by certain operations. The particular data structure chosen largely depends on the frequency of the operation that needs to be performed on the data structure.

- Traversing
- Searching
- Insertion
- Deletion
- Sorting
- Merging

C++ Syntax

Syntax refers to the rules and regulations for writing statements in a programming language.

They can also be viewed as the grammatical rules defining the structure of a programming language.

The C++ language also has its syntax for the functionalities it provides. Different statements have different syntax specifying their usage, but C++ programs also have basic syntax rules that are followed throughout all the programs.

We can learn about basic C++ Syntax using the following program.

```
#include <iostream>
// Standard Namespace
using namespace std;
// Main Function
int main() {
    // Declaration of Variable
    int num1 = 24;
    int num2 = 34;
    int result = num1 + num2;
    // Output
    cout << result << endl;
    // Return Statement
    return 0;
}
```

Output

58

The program above shows the basic C++ program that contains header files, main function, namespace declaration, etc. Let's try to understand them one by one.

a) Header File

The header files contain the definition of the functions and macros we are using in our program. In **line #1**, we used the **#include <iostream>** statement to tell the compiler to include an **iostream** header file library which stores the definition of the cin and cout standard input/output streams that we have used for input and output. **#include** is a preprocessor directive using which we import header files.

b) Namespace

A namespace in C++ is used to provide a scope or a region where we define identifiers. In **line #2**, we have used the **using namespace std** statement for specifying that we will be the standard namespace where all the standard library functions are defined.

c) Main Function

In **line #3**, we defined the main function as **int main().** The main function is the most important part of any C++ program. The program execution always starts from the main function. All the other functions are called from the main function. In C++, the main function is required to return some value indicating the execution status.

d) Blocks

Blocks are the group of statements that are enclosed within **{ }** braces. The body of the main function is from **line #4** to **line #9** enclosed within **{ }.**

e) Semicolons

As you may have noticed by now, each statement in the above code is followed by a (**;**) semicolon symbol. It is used to terminate each line of the statement of the program.

f) Identifiers

We use identifiers for the naming of variables, functions, and other user-defined data types. An identifier may consist of uppercase and lowercase alphabetical characters, underscore, and digits. The first letter must be an underscore or an alphabet.

g) Keywords

In the C++ programming language, some reserved words are used for some special meaning in the C++ program. It can't be used for identifiers. For example, the words **int, return, and using** are some keywords used in our program.

h) Basic Output cout

In line #7, we have used the 12 cout stream object (declared in the <iostream> header file) to print the sum of two numbers to the standard output stream (stdout).

i) Comments

You can see in the code there are two forward slashes (//) with some text, which are known as comments in a C++ program. Comments provide information about the code or a specific line of code to improve readability.

Arrays

What is an Array?

An array is a type of linear data structure that is defined as a collection of elements with same or different data types. They exist in both single dimension and multiple dimensions. These data structures come into picture when there is a necessity to store multiple elements of similar nature together at one place.

Memory Address	2391	2392	2393	2394	2395
Array Values	12	34	68	77	43
Array Index	0	1	2	3	4

The difference between an array index and a memory address is that the array index acts like a key value to label the elements in the array. However, a memory address is the starting address of free memory available.

Following are the important terms to understand the concept of Array.

- Element − Each item stored in an array is called an element.
- Index − Each location of an element in an array has a numerical index, which is used to identify the element.

Syntax

Creating an array in C and C++ programming languages −

data_type array_name[array_size]={elements separated by commas}

or,

data_type array_name[array_size];

Creating an array in Java programming language −

data_type[] array_name = {elements separated by commas}

or,

data_type array_name = new data_type[array_size];

Advertisement

Need for Arrays

Arrays are used as solutions to many problems from the small sorting problems to more complex problems like travelling salesperson problem. There are many data structures other than arrays that provide efficient time and space complexity for these problems, so what makes using arrays better? The answer lies in the random access lookup time.

Arrays provide O(1) random access lookup time. That means, accessing the 1st index of the array and the 1000th index of the array will both take the same time. This is due to the fact that array

comes with a pointer and an offset value. The pointer points to the right location of the memory and the offset value shows how far to look in the said memory.

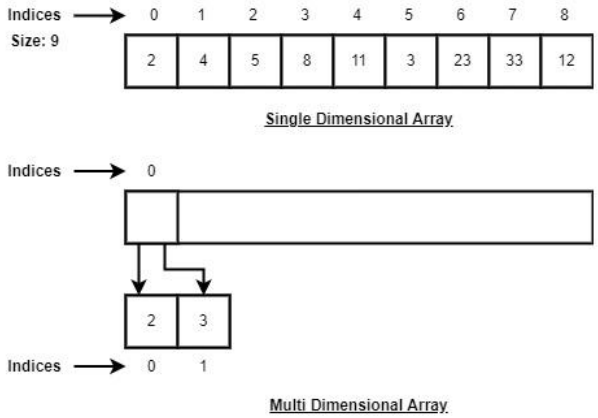
```
array_name[index]
  |      |
Pointer   Offset
```

Therefore, in an array with 6 elements, to access the 1st element, array is pointed towards the 0th index. Similarly, to access the 6th element, array is pointed towards the 5th index.

Array Representation

Arrays are represented as a collection of buckets where each bucket stores one element. These buckets are indexed from '0' to 'n-1', where n is the size of that particular array. For example, an array with size 10 will have buckets indexed from 0 to 9.

This indexing will be similar for the multidimensional arrays as well. If it is a 2-dimensional array, it will have sub-buckets in each bucket. Then it will be indexed as array_name[m][n], where m and n are the sizes of each level in the array.



As per the above illustration, following are the important points to be considered.

- Index starts with 0.
- Array length is 9 which means it can store 9 elements.
- Each element can be accessed via its index. For example, we can fetch an element at index 6 as 23.

Basic Operations in Arrays

The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update. These operations are usually performed to either modify the data in the array or to report the status of the array.

Following are the basic operations supported by an array.

- Traverse − print all the array elements one by one.
- Insertion − Adds an element at the given index.
- Deletion − Deletes an element at the given index.
- Search − Searches an element using the given index or by the value.
- Update − Updates an element at the given index.
- Display − Displays the contents of the array.

In C, when an array is initialized with size, then it assigns defaults values to its elements in following order.

Data Type	**Default Value**
bool	false
char	0
int	0
float	0.0
double	0.0f
void	
wchar_t	0

Array - Insertion Operation

In the insertion operation, we are adding one or more elements to the array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. This is done using input statements of the programming languages.

Algorithm

Following is an algorithm to insert elements into a Linear Array until we reach the end of the array –

1. Start
2. Create an Array of a desired datatype and size.

3. Initialize a variable 'i' as 0.
4. Enter the element at ith index of the array.
5. Increment i by 1.
6. Repeat Steps 4 & 5 until the end of the array.
7. Stop

Implementation

```
#include <iostream>
using namespace std;
int main(){
  int LA[3] = {}, i;
  cout << "Array Before Insertion:" << endl;
  for(i = 0; i < 3; i++)
    cout << "LA[" << i <<"] = " << LA[i] << endl;

  //prints garbage values
  cout << "Inserting elements.." <<endl;
  cout << "Array After Insertion:" << endl; // prints array values
  for(i = 0; i < 5; i++) {
    LA[i] = i + 2;
    cout << "LA[" << i <<"] = " << LA[i] << endl;
  }
  return 0;
}
```

Output

```
Array Before Insertion:
LA[0] = 0
LA[1] = 0
LA[2] = 0
Inserting elements..
Array After Insertion:
LA[0] = 2
```

LA[1] = 3

LA[2] = 4

LA[3] = 5

LA[4] = 6

Array - Deletion Operation

In this array operation, we delete an element from the particular index of an array. This deletion operation takes place as we assign the value in the consequent index to the current index.

Algorithm

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to delete an element available at the Kth position of LA.

1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set LA[J] = LA[J + 1]
5. Set J = J+1
6. Set N = N-1
7. Stop

Implementation

```
#include <iostream>
using namespace std;
int main(){
  int LA[] = {1,3,5};
  int i, n = 3;
  cout << "The original array elements are :"<<endl;
  for(i = 0; i<n; i++) {
    cout << "LA[" << i << "] = " << LA[i] << endl;
  }
  for(i = 1; i<n; i++) {
    LA[i] = LA[i+1];
    n = n - 1;
  }
```

```
  cout << "The array elements after deletion :"<<endl;
  for(i = 0; i<n; i++) {
    cout << "LA[" << i << "] = " << LA[i] <<endl;
  }
}
```

Output

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

The array elements after deletion :

LA[0] = 1

LA[1] = 5

Array - Search Operation

Searching an element in the array using a key; The key element sequentially compares every value in the array to check if the key is present in the array or not.

Algorithm

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to find an element with a value of ITEM using sequential search.

1. Start
2. Set J = 0
3. Repeat steps 4 and 5 while J < N
4. IF LA[J] is equal ITEM THEN GOTO STEP 6
5. Set J = J +1
6. PRINT J, ITEM
7. Stop

Implementation

```
#include <iostream>
using namespace std;
int main(){
  int LA[] = {1,3,5,7,8};
```

```
   int item = 5, n = 5;
   int i = 0;
   cout << "The original array elements are : " <<endl;
   for(i = 0; i<n; i++) {
      cout << "LA[" << i << "] = " << LA[i] << endl;
   }
   for(i = 0; i<n; i++) {
      if( LA[i] == item ) {
         cout << "Found element " << item << " at position " << i+1 <<endl;
      }
   }
   return 0;
}
```

Output

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

Found element 5 at position 3

Array - Traversal Operation

This operation traverses through all the elements of an array. We use loop statements to carry this out.

Algorithm

Following is the algorithm to traverse through all the elements present in a Linear Array −

1. Start
2. Initialize an Array of certain size and datatype.
3. Initialize another variable i with 0.
4. Print the ith value in the array and increment i.
5. Repeat Step 4 until the end of the array is reached.

6. End

Implementation

```
#include <iostream>
using namespace std;
int main(){
  int LA[] = {1,3,5,7,8};
  int item = 10, k = 3, n = 5;
  int i = 0, j = n;
  cout << "The original array elements are:\n";
  for(i = 0; i<n; i++)
    cout << "LA[" << i << "] = " << LA[i] << endl;
  return 0;
}
```

Output

The original array elements are :

LA[0] = 1

LA[1] = 3

LA[2] = 5

LA[3] = 7

LA[4] = 8

Array - Update Operation

Update operation refers to updating an existing element from the array at a given index.

Algorithm

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. Following is the algorithm to update an element available at the Kth position of LA.

1. Start
2. Set LA[K-1] = ITEM
3. Stop

Implementation

```
#include <iostream>
using namespace std;
```

```
int main(){
   int LA[] = {1,3,5,7,8};
   int item = 10, k = 3, n = 5;
   int i = 0, j = n;
   cout << "The original array elements are :\n";
   for(i = 0; i<n; i++)
      cout << "LA[" << i << "] = " << LA[i] << endl;
   LA[2] = item;
   cout << "The array elements after updation are :\n";
   for(i = 0; i<n; i++)
      cout << "LA[" << i << "] = " << LA[i] << endl;
   return 0;
}
```

Output

```
The original array elements are :
LA[0] = 1
LA[1] = 3
LA[2] = 5
LA[3] = 7
LA[4] = 8
The array elements after updation :
LA[0] = 1
LA[1] = 3
LA[2] = 10
LA[3] = 7
LA[4] = 8
```

Array - Display Operation

This operation displays all the elements in the entire array using a print statement.

Algorithm

Consider LA is a linear array with N elements. Following is the algorithm to display an array elements.

1. Start
2. Print all the elements in the Array
3. Stop

Implementation

```
#include <iostream>
using namespace std;
int main(){
   int LA[] = {1,3,5,7,8};
   int n = 5;
   int i;
   cout << "The original array elements are :\n";
   for(i = 0; i<n; i++)
      cout << "LA[" << i << "] = " << LA[i] << endl;
   return 0;
}
```

Output

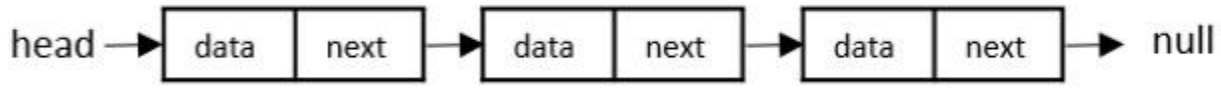
```
The original array elements are :
LA[0] = 1
LA[1] = 3
LA[2] = 5
LA[3] = 7
LA[4] = 8
```

CHAPTER 3

LINKED LIST

A linked list is a linear data structure which can store a collection of "nodes" connected together via links i.e. pointers. Linked lists nodes are not stored at a contiguous location, rather they are linked using pointers to the different memory locations. A node consists of the data value and a pointer to the address of the next node within the linked list.

A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation insertion or deletion, this helps in using system memory efficiently. Linked lists can be used to implment various data structures like a stack, queue, graph, hash maps, etc.



A linked list starts with a **head** node which points to the first node. Every node consists of data which holds the actual data (value) associated with the node and a next pointer which holds the memory address of the next node in the linked list. The last node is called the tail node in the list which points to **null** indicating the end of the list.

Linked Lists vs Arrays

In case of arrays, the size is given at the time of creation and so arrays are of fixed length where as Linked lists are dynamic in size and any number of nodes can be added in the linked lists dynamically. An array can accommodate similar types of data types where as linked lists can store various nodes of different data types.

Types of Linked List

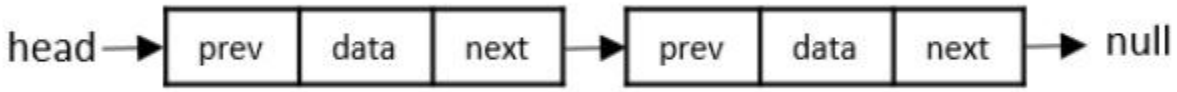
Following are the various types of linked list.

Singly Linked Lists

Singly linked lists contain two "buckets" in one node; one bucket holds the data and the other bucket holds the address of the next node of the list. Traversals can be done in one direction only as there is only a single link between two nodes of the same list.

Doubly Linked Lists

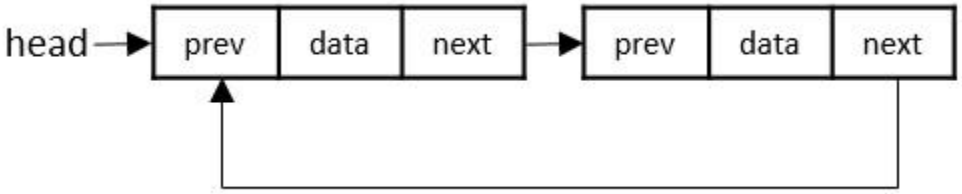
Doubly Linked Lists contain three "buckets" in one node; one bucket holds the data and the other buckets hold the addresses of the previous and next nodes in the list. The list is traversed twice as the nodes in the list are connected to each other from both sides.



Circular Linked Lists

Circular linked lists can exist in both singly linked list and doubly linked list.

Since the last node and the first node of the circular linked list are connected, the traversal in this linked list will go on forever until it is broken.



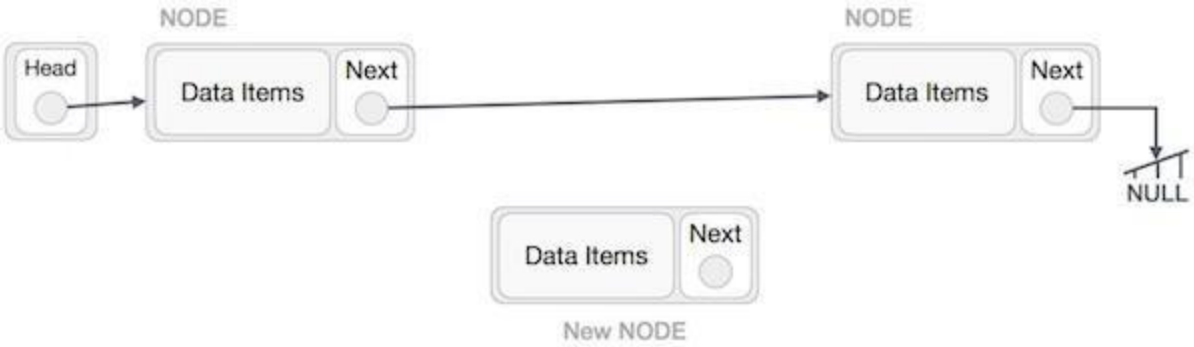
Singly linked list Basic Operations

The basic operations in the linked lists are insertion, deletion, searching, display, and deleting an element at a given key. These operations are performed on Singly Linked Lists as given below −

- **Insertion** − Adds an element at the beginning of the list.
- **Deletion** − Deletes an element at the beginning of the list.
- **Display** − Displays the complete list.
- **Search** − Searches an element using the given key.
- **Delete** − Deletes an element using the given key.

Linked List - Insertion Operation

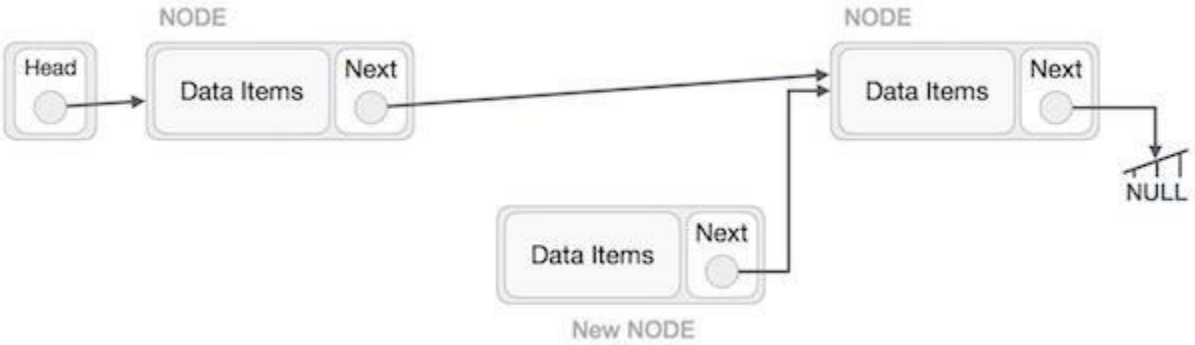
Adding a new node in linked list is a more than one step activity. We shall learn this with diagrams here. First, create a node using the same structure and find the location where it has to be inserted.



Imagine that we are inserting a node B (NewNode), between A (LeftNode) and C (RightNode). Then point B.next to C −

NewNode.next -> RightNode;

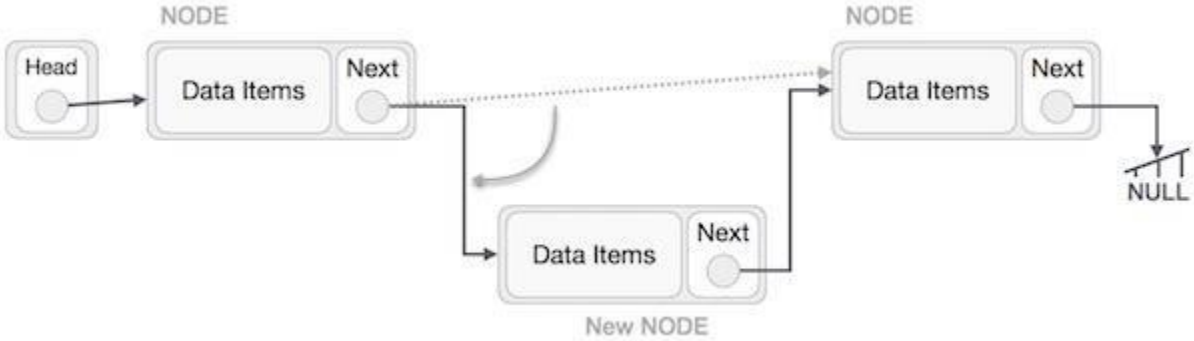
It should look like this −



Now, the next node at the left should point to the new node.

LeftNode.next -> NewNode;

This will put the new node in the middle of the two. The new list should look like this −



Insertion in linked list can be done in three different ways. They are explained as follows −

How to create a node in a linked list

1. Define the Node Structure (First Step)

Before creating a node, you must define what a node looks like.

```
struct Node {
    int data;      // Stores the value
    Node* next;    // Stores address of the next node
};
```

✓ This tells the program that **a node has data and a pointer**

2. Create a Node (Using C++ new)

Basic Way to Create a Node

```
Node* newNode = new Node;
```

What this line means

- new → allocates memory dynamically
- Node → type of data being created
- newNode → pointer that stores the address of the node

✓ Now a node exists in memory, but it has **no data yet**

3. Assign Data to the Node

```
newNode->data = 10;
```

✓ Stores 10 inside the node

4. Set the Next Pointer

```
newNode->next = NULL;
```

✓ Means this node does not point to any other node yet

✓ Usually used when the node is the **last node**

5. Complete Example: Creating One Node

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {
```

```
    Node* newNode = new Node; // Create node
    newNode->data = 25;      // Assign data
    newNode->next = NULL;    // No next node

    cout << "Node created with data: " << newNode->data;
    return 0;
}
```

6. Creating a Node and Linking It (Very Important)

```
Node* first = new Node;
first->data = 5;
first->next = NULL;

Node* second = new Node;
second->data = 10;
second->next = NULL;

first->next = second; // Link first node to second
```

✓ This creates **two nodes**

✓ Links them together to form a linked list

Insertion at Beginning

In this operation, we are adding an element at the beginning of the list.

*A **pointer** is a **variable that stores the memory address of another variable**.*

Algorithm

1. START
2. Create a node to store the data
3. Check if the list is empty
4. If the list is empty, add the data to the node and assign the head pointer to it.
5. If the list is not empty, add the data to a node and link to the current head. Assign the head to the newly added node.
6. END

Example

```
#include <iostream>      // Includes input/output library for cout and cin
using namespace std;     // Allows use of standard names without std::
// Define the structure of a node
struct Node {
    int data;           // Variable to store the data/value of the node
    Node* next;         // Pointer that stores the address of the next node
};
// Head pointer (points to the first node of the linked list)
Node* head = NULL;      // Initialize head to NULL (list is empty)
// Function to display the linked list
void printList() {
    Node* p = head;     // Create a temporary pointer and set it to head
    cout << "[ ";       // Print opening bracket
    while (p != NULL) { // Loop through the list until the end
        cout << p->data << " "; // Print data of the current node
        p = p->next;    // Move pointer to the next node This line moves the pointer p to the next node
in a linked list.
    }
    cout << "]";        // Print closing bracket
}
// Function to insert a node at the beginning of the list
void insertAtBegin(int value) {
    Node* newNode = new Node; // dynamically create a new node in memory
    newNode->data = value;   // Store the given value in the new node
    newNode->next = head;    // Make new node point to the old first node
    head = newNode;          // Update head to point to the new node
}
int main() {
    insertAtBegin(12);  // Insert 12 at the beginning of the list
    insertAtBegin(22);  // Insert 22 at the beginning
    insertAtBegin(30);  // Insert 30 at the beginning
```

```
    insertAtBegin(44);  // Insert 44 at the beginning
    insertAtBegin(50);  // Insert 50 at the beginning
    cout << "Linked List: "; // Display message before printing list
    printList();            // Call function to display the list
    return 0;               // End the program successfully
}
```

Insertion at Ending

In this operation, we are adding an element at the ending of the list.

Algorithm

1. START
2. Create a new node and assign the data
3. Find the last node
4. Point the last node to new node
5. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>          // Includes input and output library
using namespace std;         // Allows use of standard C++ names
// Define the structure of a node
struct Node {
    int data;               // Stores the data/value of the node
    Node* next;             // Pointer to the next node in the list
};
// Declare global pointers
Node* head = NULL;          // Head pointer, initially NULL (empty list)
Node* current = NULL;       // Current pointer (not used, included for similarity)
// Function to display the linked list
void printList() {
    Node* p = head;         // Temporary pointer starting from head
    cout << "\n[";          // Print opening bracket
    // Loop through the linked list
```

```
    while (p != NULL) {     // Continue until end of list
        cout << " " << p->data << " "; // Print data of current node
        p = p->next;        // Move pointer to the next node
    }
    cout << "]";            // Print closing bracket
}
// Function to insert a node at the beginning
void insertAtBegin(int data) {
    Node* lk = new Node;    // Create a new node dynamically
    lk->data = data;        // Store the given data in the node
    lk->next = head;        // Point new node to the old first node
    head = lk;              // Update head to point to the new node
}
// Function to insert a node at the end
void insertAtEnd(int data) {
    Node* lk = new Node;    // Create a new node dynamically
    lk->data = data;        // Store the given data
    lk->next = NULL;        // New node will be the last node
    Node* linkedlist = head; // Pointer to traverse the list
    // Traverse until the last node
    while (linkedlist->next != NULL)
        linkedlist = linkedlist->next;
    linkedlist->next = lk;  // Link last node to the new node
}
int main() {
    insertAtBegin(12);      // Insert 12 at the beginning
    insertAtEnd(22);        // Insert 22 at the end
    insertAtEnd(30);        // Insert 30 at the end
    insertAtEnd(44);        // Insert 44 at the end
    insertAtEnd(50);        // Insert 50 at the end
    cout << "Linked List: "; // Display message
```

```
    printList();            // Call function to display list
    return 0;               // End program successfully
}
```

Output

Linked List:

[12 22 30 44 50]

Insertion at a Given Position

In this operation, we are adding an element at any position within the list.

Algorithm

1. START
2. Create a new node and assign data to it
3. Iterate until the node at position is found
4. Point first to new first node
5. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>          // Include input and output library
using namespace std;         // Allows use of cout without std::
// Define the structure of a node
struct Node {
    int data;               // Stores the data value of the node
    Node* next;             // Pointer to the next node in the list
};
// Declare global pointers
Node* head = NULL;          // Head pointer, initially NULL (empty list)
Node* current = NULL;       // Current pointer (not used, included for similarity)
// Function to display the linked list
void printList() {
    Node* p = head;         // Temporary pointer starting from head
    cout << "\n[";          // Print opening bracket
    // Start traversing from the beginning
```

```
    while (p != NULL) {     // Loop until the end of the list
        cout << " " << p->data << " "; // Print data of the current node
        p = p->next;        // Move pointer to the next node
    }
    cout << "]";            // Print closing bracket
}
// Function to insert a node at the beginning
void insertAtBegin(int data) {
    Node* lk = new Node;    // Create a new node dynamically
    lk->data = data;        // Store the given data in the new node
    lk->next = head;        // Point new node to the old first node
    head = lk;              // Update head to point to the new node
}
// Function to insert a node after a given node
void insertAfterNode(Node* list, int data) {
    Node* lk = new Node;    // Create a new node dynamically
    lk->data = data;        // Store the given data in the node
    lk->next = list->next; // Make new node point to the next node
    list->next = lk;        // Link the given node to the new node
}
// Main function
int main() {
    int k = 0;              // Integer variable (not used, kept for similarity)
    insertAtBegin(12);      // Insert 12 at the beginning
    insertAtBegin(22);      // Insert 22 at the beginning
    insertAfterNode(head->next, 30); // Insert 30 after the second node
    cout << "Linked List: "; // Display message
    printList();            // Call function to print the linked list
    return 0;               // End the program successfully
}
Linked List:
```

[22 12 30]

Linked List - Deletion Operation

Deletion is also a more than one-step process. We shall learn with pictorial representation. First, locate the target node to be removed, by using searching algorithms.

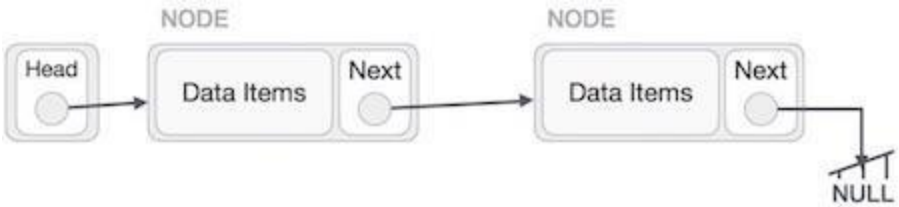
The left (previous) node of the target node now should point to the next node of the target node −

LeftNode.next -> TargetNode.next;

This will remove the link that was pointing to the target node. Now, using the following code, we will remove what the target node is pointing at.

TargetNode.next -> NULL;

We need to use the deleted node. We can keep that in memory otherwise we can simply deallocate memory and wipe off the target node completely.



Similar steps should be taken if the node is being inserted at the beginning of the list. While inserting it at the end, the second last node of the list should point to the new node and the new node will point to NULL.

Deletion in linked lists is also performed in three different ways. They are as follows −

Deletion at Beginning

In this deletion operation of the linked, we are deleting an element from the beginning of the list. For this, we point the head to the second node.

Algorithm

1. START
2. Assign the head pointer to the next node in the list
3. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>          // Include input and output library
using namespace std;         // Allows use of standard C++ names like cout
// Define the structure of a node
struct Node {
   int data;              // Stores the value/data of the node
   Node* next;            // Pointer to the next node in the linked list
};
// Declare global pointers
Node* head = NULL;         // Head pointer initialized to NULL (empty list)
Node* current = NULL;      // Current pointer (not used, kept for similarity)
```

```
// Function to display the linked list
void printList() {
    Node* p = head;         // Create a temporary pointer starting from head
    cout << "\n[";          // Print opening square bracket
    // Start traversing from the beginning
    while (p != NULL) {     // Loop until the end of the list
        cout << " " << p->data << " "; // Print data of current node
        p = p->next;        // Move pointer to the next node
    }
    cout << "]";            // Print closing square bracket
}
// Function to insert a node at the beginning
void insertAtBegin(int data) {
    Node* lk = new Node;    // Dynamically create a new node
    lk->data = data;        // Store the given data in the new node
    lk->next = head;        // Point new node to the old first node
    head = lk;              // Update head to point to the new node
}
// Function to delete a node at the beginning
void deleteAtBegin() {
    if (head != NULL) {     // Check if the list is not empty
        head = head->next;  // Move head to the next node (first node removed)
    }
}
// Main function
int main() {
    int k = 0;              // Integer variable (not used, kept for similarity)
    insertAtBegin(12);      // Insert 12 at the beginning
    insertAtBegin(22);      // Insert 22 at the beginning
    insertAtBegin(30);      // Insert 30 at the beginning
    insertAtBegin(40);      // Insert 40 at the beginning
```

```
    insertAtBegin(55);      // Insert 55 at the beginning
    cout << "Linked List: "; // Display message
    printList();            // Print the linked list
    deleteAtBegin();        // Delete the first node
    cout << "\nLinked List after deletion: "; // Display message
    printList();            // Print the updated linked list
    return 0;               // End the program successfully
}
```

Output

Linked List:

[55 40 30 22 12]

Linked List after deletion:

[40 30 22 12]

Deletion at Ending

In this deletion operation of the linked, we are deleting an element from the ending of the list.

Algorithm

1. START
2. Iterate until you find the second last element in the list.
3. Assign NULL to the second last element in the list.
4. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>         // Include input and output library
using namespace std;        // Allows use of standard C++ names like cout
// Define the structure of a node
struct Node {
    int data;               // Stores the value/data of the node
    Node* next;             // Pointer to the next node in the linked list
};
// Declare global pointers
Node* head = NULL;          // Head pointer initialized to NULL (empty list)
```

```
Node* current = NULL;       // Current pointer (not used, kept for similarity)
// Function to display the linked list
void printList() {
    Node* p = head;         // Create a temporary pointer starting from head
    cout << "\n[";          // Print opening square bracket
    // Start traversing from the beginning
    while (p != NULL) {     // Loop until the end of the list
        cout << " " << p->data << " "; // Print data of current node
        p = p->next;        // Move pointer to the next node
    }
    cout << "]";            // Print closing square bracket
}
// Function to insert a node at the beginning
void insertAtBegin(int data) {
    Node* lk = new Node;    // Dynamically create a new node
    lk->data = data;        // Store the given data in the new node
    lk->next = head;        // Point new node to the old first node
    head = lk;              // Update head to point to the new node
}
// Function to delete a node at the end
void deleteAtEnd() {
    if (head == NULL)       // Check if the list is empty
        return;             // Exit if there is nothing to delete
    if (head->next == NULL) { // Check if there is only one node
        head = NULL;        // Remove the only node
        return;             // Exit the function
    }
    Node* linkedlist = head; // Temporary pointer for traversal
    // Traverse until the second-last node
    while (linkedlist->next->next != NULL) {
        linkedlist = linkedlist->next; // Move to next node
```

```
    }
    linkedlist->next = NULL; // Remove the last node
}
// Main function
int main() {
    int k = 0;              // Integer variable (not used, kept for similarity)
    insertAtBegin(12);      // Insert 12 at the beginning
    insertAtBegin(22);      // Insert 22 at the beginning
    insertAtBegin(30);      // Insert 30 at the beginning
    insertAtBegin(40);      // Insert 40 at the beginning
    insertAtBegin(55);      // Insert 55 at the beginning

    cout << "Linked List: "; // Display message
    printList();            // Print the linked list
    deleteAtEnd();          // Delete the last node
    cout << "\nLinked List after deletion: "; // Display message
    printList();            // Print the updated linked list
    return 0;               // End the program successfully
}
```

Output

Linked List:

[55 40 30 22 12]

Linked List after deletion:

[55 40 30 22]

Deletion at a Given Position

In this deletion operation of the linked, we are deleting an element at any position of the list.

Algorithm

1. START
2. Iterate until find the current node at position in the list.
3. Assign the adjacent node of current node in the list to its previous node.
4. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>          // Allows input and output operations
using namespace std;         // Enables use of standard C++ names like cout
// Define the structure of a node
struct Node {
   int data;               // Stores the data value of the node
   Node* next;             // Pointer to the next node in the linked list
};
// Declare global pointers
Node* head = NULL;          // Head pointer initialized to NULL (empty list)
Node* current = NULL;       // Current pointer (not used, kept for similarity)
// Function to display the linked list
void printList() {
   Node* p = head;          // Start traversal from the head node
   cout << "\n[";           // Print opening square bracket
   // Traverse the linked list until the end
   while (p != NULL) {
      cout << " " << p->data << " "; // Print data of the current node
      p = p->next;          // Move to the next node
   }
   cout << "]";             // Print closing square bracket
}
// Function to insert a node at the beginning of the list
void insertAtBegin(int data) {
   Node* lk = new Node;      // Create a new node dynamically
   lk->data = data;          // Assign the given value to the node
   lk->next = head;          // Point new node to the current head
   head = lk;               // Update head to point to the new node
}
// Function to delete a node with a specific value (key)
```

```
void deleteNode(int key) {
    Node* temp = head;       // Temporary pointer to traverse the list
    Node* prev = NULL;       // Pointer to store the previous node
    // If the head node itself contains the key
    if (temp != NULL && temp->data == key) {
        head = temp->next;   // Move head to the next node
        delete temp;         // Free the deleted node
        return;              // Exit the function
    }
    // Search for the node with the given key
    while (temp != NULL && temp->data != key) {
        prev = temp;         // Store current node as previous
        temp = temp->next;  // Move to next node
    }
    // If the key is not found in the list
    if (temp == NULL)
        return;              // Exit the function
    // Unlink the node from the list
    prev->next = temp->next; // Bypass the node to be deleted
    delete temp;             // Free memory of the deleted node
}
// Main function
int main() {
    int k = 0;               // Integer variable (not used, kept for similarity)
    insertAtBegin(12);       // Insert 12 at the beginning
    insertAtBegin(22);       // Insert 22 at the beginning
    insertAtBegin(30);       // Insert 30 at the beginning
    insertAtBegin(40);       // Insert 40 at the beginning
    insertAtBegin(55);       // Insert 55 at the beginning
    cout << "Linked List: "; // Display message
    printList();             // Print the linked list
```

```
    deleteNode(30);          // Delete the node with value 30
    cout << "\nLinked List after deletion: "; // Display message
    printList();             // Print the updated list
    return 0;                // End the program successfully
}
```

Output

Linked List:

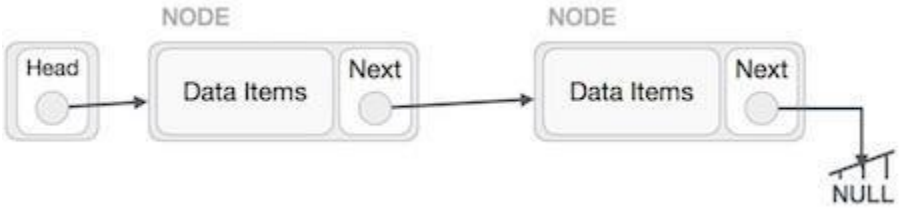
[55 40 30 22 12]

Linked List after deletion:

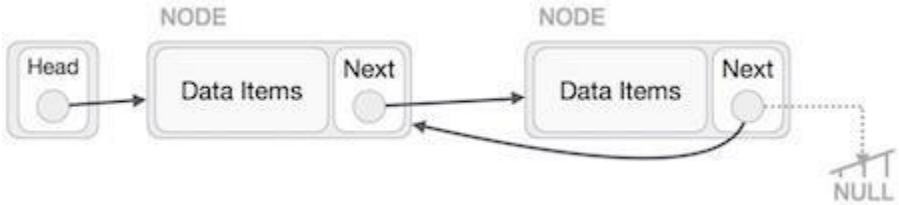
[55 40 22 12]

Linked List - Reversal Operation

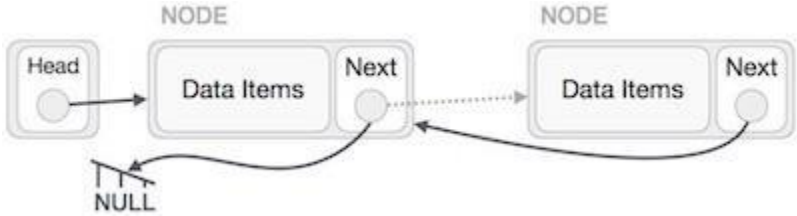
This operation is a thorough one. We need to make the last node to be pointed by the head node and reverse the whole linked list.



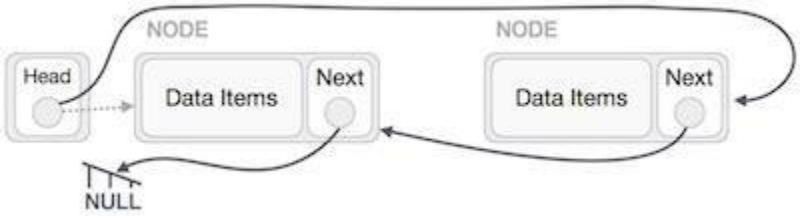
First, we traverse to the end of the list. It should be pointing to NULL. Now, we shall make it point to its previous node −



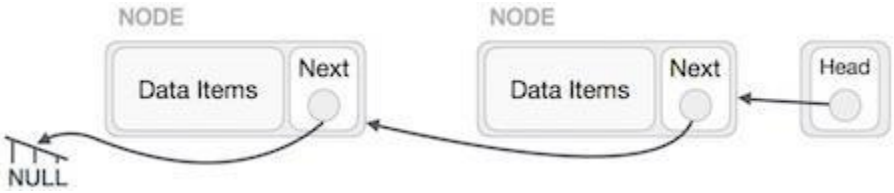
We have to make sure that the last node is not the last node. So we'll have some temp node, which looks like the head node pointing to the last node. Now, we shall make all left side nodes point to their previous nodes one by one.



Except the node (first node) pointed by the head node, all nodes should point to their predecessor, making them their new successor. The first node will point to NULL.



We'll make the head node point to the new first node by using the temp node.



Algorithm

Step by step process to reverse a linked list is as follows −

1. START
2. We use three pointers to perform the reversing: prev, next, head.
3. Point the current node to head and assign its next value to the prev node.
4. Iteratively repeat the step 3 for all the nodes in the list.
5. Assign head to the prev node.

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>            // Enables input and output operations
using namespace std;           // Allows use of cout without std::
// Define the structure of a node
struct Node {
   int data;                   // Stores the data value of the node
   Node* next;                 // Pointer to the next node in the list
};
// Declare global pointers
Node* head = NULL;             // Head pointer initialized to NULL
Node* current = NULL;          // Current pointer (not used, kept for similarity)
// Function to display the linked list
```

```
void printList() {
    Node* p = head;             // Temporary pointer starting at head
    cout << "\n[";              // Print opening bracket
    // Traverse the list from beginning to end
    while (p != NULL) {
        cout << " " << p->data << " "; // Print data in current node
        p = p->next;            // Move to the next node
    }
    cout << "]";                // Print closing bracket
}
// Function to insert a node at the beginning of the list
void insertAtBegin(int data) {
    Node* lk = new Node;        // Dynamically create a new node
    lk->data = data;            // Store the given data in the node
    lk->next = head;            // Point new node to the old head
    head = lk;                  // Update head to point to the new node
}


// Function to reverse the linked list
void reverseList(Node** head) {
    Node* prev = NULL;          // Pointer to store previous node
    Node* cur = *head;          // Pointer to current node (start from head)
    Node* tmp;                  // Temporary pointer to store next node
    // Traverse the list until the end
    while (cur != NULL) {
        tmp = cur->next;        // Store address of next node
        cur->next = prev;       // Reverse the link
        prev = cur;             // Move prev one step forward
        cur = tmp;              // Move cur one step forward
    }
```

```
    *head = prev;              // Update head to the last node (new head)
}
// Main function
int main() {
    int k = 0;                 // Integer variable (not used)
    insertAtBegin(12);         // Insert 12 at the beginning
    insertAtBegin(22);         // Insert 22 at the beginning
    insertAtBegin(30);         // Insert 30 at the beginning
    insertAtBegin(40);         // Insert 40 at the beginning
    insertAtBegin(55);         // Insert 55 at the beginning
    cout << "Linked List: ";    // Display message
    printList();               // Print original linked list
    reverseList(&head);        // Reverse the linked list
    cout << "\nReversed Linked List: "; // Display message
    printList();               // Print reversed linked list
    return 0;                  // End the program
}
```

Output

Linked List:

[55 40 30 22 12]

Reversed Linked List:

[12 22 30 40 55]

Linked List - Search Operation

Searching for an element in the list using a key element. This operation is done in the same way as array search; comparing every element in the list with the key element given.

Algorithm

1. 1 START
2. 2 If the list is not empty, iteratively check if the list contains the key
3. 3 If the key element is not present in the list, unsuccessful search
4. 4 END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>             // Enables input and output operations
using namespace std;            // Allows use of standard names like cout
// Define the structure of a node
struct Node {
   int data;                    // Stores the value of the node
   Node* next;                  // Pointer to the next node in the list
};
// Declare global pointers
Node* head = NULL;              // Head pointer initialized to NULL (empty list)
Node* current = NULL;           // Current pointer (not used, kept for similarity)
// Function to display the linked list
void printList() {
   Node* p = head;              // Start traversal from the head node
   cout << "\n[";               // Print opening bracket
   // Traverse the list from beginning to end
   while (p != NULL) {
      cout << " " << p->data << " "; // Print data of current node
      p = p->next;              // Move to the next node
   }
   cout << "]";                 // Print closing bracket
}
// Function to insert a node at the beginning of the list
void insertAtBegin(int data) {
   Node* lk = new Node;         // Dynamically create a new node
   lk->data = data;             // Store the given data in the node
   lk->next = head;             // Point new node to the old head
   head = lk;                   // Update head to point to the new node
}
// Function to search for an element in the linked list
int searchList(int key) {
```

```
    Node* temp = head;         // Temporary pointer to traverse the list
    // Traverse the list until the end
    while (temp != NULL) {
        if (temp->data == key) // Check if current node contains the key
            return 1;          // Return 1 if element is found
        temp = temp->next;     // Move to the next node
    }
    return 0;                  // Return 0 if element is not found
}
// Main function
int main() {
    int k = 0;                 // Variable to store search result
    insertAtBegin(12);         // Insert 12 at the beginning
    insertAtBegin(22);         // Insert 22 at the beginning
    insertAtBegin(30);         // Insert 30 at the beginning
    insertAtBegin(40);         // Insert 40 at the beginning
    insertAtBegin(55);         // Insert 55 at the beginning
    cout << "Linked List: ";   // Display message
    printList();               // Print the linked list
    int ele = 30;              // Element to be searched
    cout << "\nElement to be searched is: " << ele;
    k = searchList(ele);       // Call search function
    if (k == 1)                // If search returns 1
        cout << "\nElement is found";
    else                       // If search returns 0
        cout << "\nElement is not found in the list";
    return 0;                  // End the program
}
```

Output

```
Linked List:
[ 55  40  30  22  12 ]
```

Element to be searched is: 30

Element is found

Linked List - Traversal Operation

The traversal operation walks through all the elements of the list in an order and displays the elements in that order.

Algorithm

1. START
2. While the list is not empty and did not reach the end of the list, print the data in each node
3. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>            // Enables input and output operations
using namespace std;           // Allows use of cout without std::
// Define the structure of a node
struct Node {
   int data;                 // Stores the value/data of the node
   Node* next;               // Pointer to the next node in the linked list
};



// Declare global pointers
Node* head = NULL;            // Head pointer initialized to NULL (empty list)
Node* current = NULL;         // Current pointer (not used, kept for similarity)
// Function to display the linked list
void printList() {
   Node* p = head;            // Create a pointer and set it to head
   cout << "\n[";             // Print opening bracket
   // Start traversing from the beginning
   while (p != NULL) {
      cout << " " << p->data << " "; // Print data of the current node
      p = p->next;           // Move to the next node
```

```
    }
    cout << "]";              // Print closing bracket
}
// Function to insert a node at the beginning of the list
void insertAtBegin(int data) {
    Node* lk = new Node;       // Dynamically create a new node
    lk->data = data;           // Assign the given data to the node
    lk->next = head;           // Point the new node to the old head
    head = lk;                 // Update head to point to the new node
}
// Main function
int main() {
    int k = 0;                 // Integer variable (not used)
    insertAtBegin(12);         // Insert 12 at the beginning
    insertAtBegin(22);         // Insert 22 at the beginning
    insertAtBegin(30);         // Insert 30 at the beginning
    cout << "Linked List: ";   // Display message
    printList();               // Print the linked list
    return 0;                  // End the program
}
```

Output

Linked List:

[30 22 12]

Linked List - Complete implementation

Following are the complete implementations of Linked List in various programming languages –

```
#include <iostream>            // For input/output operations
using namespace std;           // Allows use of cout and endl without std::


// Define the structure of a node
struct Node {
```

```
    int data;                   // Stores the value of the node
    Node* next;                 // Pointer to the next node in the list
};
// Declare global pointers
Node* head = NULL;              // Head pointer, points to the first node
Node* current = NULL;           // Current pointer (not used here, optional)
// Function to display the linked list
void printList() {
    Node* p = head;             // Temporary pointer to traverse the list
    cout << "\n[";              // Print opening bracket
    // Traverse from head to end
    while (p != NULL) {
        cout << " " << p->data << " "; // Print current node's data
        p = p->next;                   // Move to next node
    }
    cout << "]";                // Print closing bracket
}
// Function to insert a node at the beginning
void insertAtBegin(int data) {
    Node* lk = new Node;        // Create a new node dynamically
    lk->data = data;            // Assign the value
    lk->next = head;            // Link new node to old head
    head = lk;                  // Update head to point to new node
}
// Function to insert a node at the end
void insertAtEnd(int data) {
    Node* lk = new Node;        // Create a new node dynamically
    lk->data = data;            // Assign the value
    lk->next = NULL;            // Last node points to NULL
    if (head == NULL) {         // If list is empty
        head = lk;              // New node becomes head
```

```
        return;
    }
    Node* linkedlist = head;    // Temporary pointer to traverse
    while (linkedlist->next != NULL)  // Traverse to last node
        linkedlist = linkedlist->next;
    linkedlist->next = lk;      // Link last node to new node
}


// Function to insert a node after a given node
void insertAfterNode(Node* list, int data) {
    Node* lk = new Node;        // Create new node
    lk->data = data;            // Assign data
    lk->next = list->next;      // Link new node to next node
    list->next = lk;            // Link previous node to new node
}
// Function to delete the first node
void deleteAtBegin() {
    if (head != NULL) {
        Node* temp = head;      // Store head node
        head = head->next;      // Move head to next node
        delete temp;            // Free memory of old head
    }
}
// Function to delete the last node
void deleteAtEnd() {
    if (head == NULL) return;   // Empty list
    if (head->next == NULL) {   // Only one node
        delete head;            // Delete head
        head = NULL;            // List becomes empty
        return;
```

```
    }

    Node* linkedlist = head;    // Temporary pointer
    while (linkedlist->next->next != NULL) // Traverse to second-last node
        linkedlist = linkedlist->next;
    delete linkedlist->next;    // Delete last node
    linkedlist->next = NULL;    // Update second-last node to NULL
}
// Function to delete a node by value (key)
void deleteNode(int key) {
    Node* temp = head;
    Node* prev = NULL;
    // If head contains the key
    if (temp != NULL && temp->data == key) {
        head = temp->next;      // Move head
        delete temp;            // Free memory
        return;
    }
    // Traverse to find the key
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) return;   // Key not found
    prev->next = temp->next;    // Unlink the node
    delete temp;                // Free memory
}
// Function to search an element in the list
int searchList(int key) {
    Node* temp = head;          // Temporary pointer
```

```
    while (temp != NULL) {
        if (temp->data == key)  // Check if current node matches key
            return 1;           // Element found
        temp = temp->next;      // Move to next node
    }
    return 0;                   // Element not found
}
// Main function
int main() {
    int k = 0;                  // Variable to store search result
    // Insert nodes at beginning and end
    insertAtBegin(12);
    insertAtBegin(22);
    insertAtEnd(30);
    insertAtEnd(44);
    insertAtBegin(50);
    // Insert 33 after the third node
    insertafternode(head->next->next, 33);
    cout << "Linked List: ";    // Display message
    printList();                // Print current list
    // Perform deletions
    deleteAtBegin();            // Delete first node
    deleteAtEnd();              // Delete last node
    deleteNode(12);             // Delete node with value 12
    cout << "\nLinked List after deletion: ";
    printList();                // Print list after deletions
    // Insert more nodes
    insertAtBegin(4);
    insertAtBegin(16);
    cout << "\nUpdated Linked List: ";
    printList();                // Print updated list
```

```
    // Search for element
    k = searchList(16);
    if (k == 1)
        cout << "\nElement is found";
    else
        cout << "\nElement is not present in the list";
    return 0;                   // End program
}
```

Output

Linked List:

[50 22 12 33 30 44]

Linked List after deletion:

[22 33 30]

Updated Linked List:

[16 4 22 33 30]

Element is found

Doubly Linked

Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, forward as well as backward easily as compared to Single Linked List. Following are the important terms to understand the concept of doubly linked list.

Link − each link of a linked list can store a data called an element.

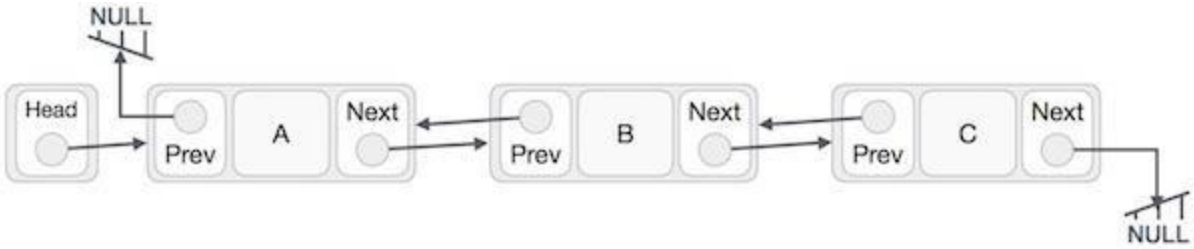
Next − each link of a linked list contains a link to the next link called Next.

Prev − each link of a linked list contains a link to the previous link called Prev.

Linked List − A Linked List contains the connection link to the first link called First and to the last link called Last.

Advertisement

Doubly Linked List Representation



As per the above illustration, following are the important points to be considered.

Doubly Linked List contains a link element called first and last.

Each link carries a data field(s) and a link field called next.

Each link is linked with its next link using its next link.

Each link is linked with its previous link using its previous link.

The last link carries a link as null to mark the end of the list.

Program to create a doubly linked list

```
#include <iostream>     // Allows input and output using cin and cout
using namespace std;    // Allows us to use cout without std::
/*
  Structure definition for a node in a doubly linked list
*/
struct Node {
   int data;         // Stores the data value of the node
   Node* prev;       // Pointer to the previous node
   Node* next;       // Pointer to the next node
};
// Head pointer that always points to the first node in the list
Node* head = nullptr; // points to nothing
/*
  Function to insert a node at the beginning of the doubly linked list
*/
void insertAtBeginning(int value) {
   Node* newNode = new Node();// Create a new node dynamically in memory
   newNode->data = value; // Assign the given value to the data field of the new node
   newNode->prev = nullptr; // Since this node will be the first node, previous pointer is NULL
```

```
    newNode->next = head; // Make the next pointer of the new node point to the current head
    // If the list is not empty
    if (head != nullptr) {
        // Set the previous pointer of the old head to the new node
        head->prev = newNode;
    }
    // Move the head pointer to point to the new node
    head = newNode;
}
/*
   Function to display the doubly linked list from first to last
*/
void display() {
    // Create a temporary pointer to traverse the list
    Node* temp = head;
    // Traverse the list until the end is reached
    while (temp != nullptr) {
        // Print the data of the current node
        cout << temp->data << " <-> ";
        // Move to the next node
        temp = temp->next;
    }
    // Indicate the end of the list
    cout << "NULL" << endl;
}
/*
   Main function - program execution starts here
*/
int main() {
    // Insert 10 at the beginning of the list
    insertAtBeginning(10);
```

```
    // Insert 20 at the beginning of the list
    insertAtBeginning(20);
    // Insert 30 at the beginning of the list
    insertAtBeginning(30);
    // Display the contents of the doubly linked list
    cout << "Doubly Linked List:" << endl;
    display();
    // Indicate successful program termination
    return 0;
}
```

Basic Operations in Doubly Linked List

Following are the basic operations supported by a list.

Insertion − Adds an element at the beginning of the list.

Insert Last − Adds an element at the end of the list.

Insert After − Adds an element after an item of the list.

Deletion − Deletes an element at the beginning of the list.

Delete Last − Deletes an element from the end of the list.

Delete − Deletes an element from the list using the key.

Display forward − Displays the complete list in a forward manner.

Display backward − Displays the complete list in a backward manner.

Doubly Linked List - Insertion at the Beginning

In this operation, we create a new node with three compartments, one containing the data, the others containing the address of its previous and next nodes in the list. This new node is inserted at the beginning of the list.

Algorithm

1. START
2. Create a new node with three variables: prev, data, next.
3. Store the new data in the data variable
4. If the list is empty, make the new node as head.
5. Otherwise, link the address of the existing first node to the next variable of the new node, and assign null to the prev variable.

6. Point the head to the new node.
7. END

Doubly Linked List - Insertion at the End

In this insertion operation, the new input node is added at the end of the doubly linked list; if the list is not empty. The head will be pointed to the new node, if the list is empty.

Algorithm

1. START
2. If the list is empty, add the node to the list and point the head to it.
3. If the list is not empty, find the last node of the list.
4. Create a link between the last node in the list and the new node.
5. The new node will point to NULL as it is the new last node.
6. END

Doubly Linked List - Deletion at the Beginning

This deletion operation deletes the existing first nodes in the doubly linked list. The head is shifted to the next node and the link is removed.

Algorithm

1. START
2. Check the status of the doubly linked list
3. If the list is empty, deletion is not possible
4. If the list is not empty, the head pointer is shifted to the next node.
5. END

Doubly Linked List - Complete Implementation

```
#include <iostream>     // Enables input and output using cout
using namespace std;    // Allows use of standard names without std:: prefix
// Structure representing a node in a doubly linked list
struct Node {
   int data;          // Stores the data value
   int key;           // Stores the key value
   Node* next;        // Pointer to the next node
   Node* prev;        // Pointer to the previous node
};
```

```
// Pointer that always points to the first node
Node* head = nullptr;
// Pointer that always points to the last node
Node* last = nullptr;
// Function to check if the list is empty
bool isEmpty() {
    return head == nullptr;   // Returns true if list has no nodes
}
// Display the list from first node to last node
void displayForward() {
    Node* ptr = head;         // Start from the head
    cout << "\n[ ";
    while (ptr != nullptr) {  // Traverse until end of list
        cout << "(" << ptr->key << "," << ptr->data << ") ";
        ptr = ptr->next;      // Move forward
    }
    cout << "]" << endl;
}
// Display the list from last node to first node
void displayBackward() {
    Node* ptr = last;         // Start from the last node
    cout << "\n[ ";
    while (ptr != nullptr) {  // Traverse backwards
        cout << "(" << ptr->key << "," << ptr->data << ") ";
        ptr = ptr->prev;      // Move backward
    }
    cout << "]" << endl;
}
// Insert a node at the beginning of the list
void insertFirst(int key, int data) {
    // Create a new node
```

```
    Node* link = new Node();
    link->key = key;
    link->data = data;
    link->prev = nullptr;     // First node has no previous
    // If the list is empty
    if (isEmpty()) {
        link->next = nullptr;
        last = link;          // New node is also the last node
    } else {
        link->next = head;    // Point new node to old head
        head->prev = link;    // Old head points back to new node
    }
    head = link;              // Update head to new node
}
// Insert a node at the end of the list
void insertLast(int key, int data) {
    // Create a new node
    Node* link = new Node();
    link->key = key;
    link->data = data;
    link->next = nullptr;     // Last node has no next
    // If the list is empty
    if (isEmpty()) {
        link->prev = nullptr;
        head = link;          // New node becomes the head
    } else {
        link->prev = last;    // Link back to old last
        last->next = link;    // Old last points to new node
    }
    last = link;              // Update last pointer
}
```

```
// Delete the first node in the list
Node* deleteFirst() {
    Node* temp = head;        // Store reference to head
    // If only one node exists
    if (head->next == nullptr) {
        head = nullptr;
        last = nullptr;
    } else {
        head = head->next;    // Move head forward
        head->prev = nullptr;
    }
    return temp;              // Return deleted node
}
// Delete the last node in the list
Node* deleteLast() {
    Node* temp = last;        // Store reference to last node
    // If only one node exists
    if (head->next == nullptr) {
        head = nullptr;
        last = nullptr;
    } else {
        last = last->prev;    // Move last backward
        last->next = nullptr;
    }
    return temp;              // Return deleted node
}
// Delete a node with a specific key
Node* deleteNode(int key) {
    Node* current = head;

    // Traverse the list to find the key
```

```
    while (current != nullptr && current->key != key) {
        current = current->next;
    }
    // If key not found
    if (current == nullptr) {
        return nullptr;
    }
    // If node is head
    if (current == head) {
        head = current->next;
    } else {
        current->prev->next = current->next;
    }
    // If node is last
    if (current == last) {
        last = current->prev;
    } else {
        current->next->prev = current->prev;
    }
    return current;          // Return deleted node
}
// Insert a node after a given key
bool insertAfter(int key, int newKey, int data) {
    Node* current = head;
    // Traverse list to find the given key
    while (current != nullptr && current->key != key) {
        current = current->next;
    }
    // If key not found
    if (current == nullptr) {
        return false;
```

```
    }
    // Create new node
    Node* newLink = new Node();
    newLink->key = newKey;
    newLink->data = data;
    // If inserting after the last node
    if (current == last) {
        newLink->next = nullptr;
        newLink->prev = last;
        last->next = newLink;
        last = newLink;
    } else {
        newLink->next = current->next;
        newLink->prev = current;
        current->next->prev = newLink;
        current->next = newLink;
    }
    return true;
}
// Main function
int main() {
    // Insert nodes at the beginning
    insertFirst(1, 10);
    insertFirst(2, 20);
    insertFirst(3, 30);
    insertFirst(4, 1);
    insertFirst(5, 40);
    insertFirst(6, 56);
    cout << "List (First to Last):";
    displayForward();
    cout << "List (Last to First):";
```

```
    displayBackward();
    cout << "After deleting first node:";
    deleteFirst();
    displayForward();
    cout << "After deleting last node:";
    deleteLast();
    displayForward();
    cout << "After inserting after key 4:";
    insertAfter(4, 7, 13);
    displayForward();
    cout << "After deleting key 4:";
    deleteNode(4);
    displayForward();
    return 0;   // End of program
}
```

Output

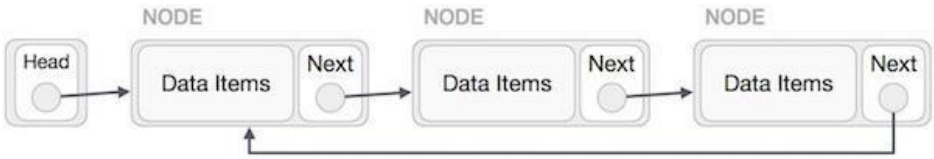
```
List (First to Last):
[ (6, 56) (5, 40) (4, 1) (3, 30) (2, 20) (1, 10) ]
List (Last to First):
[ (1, 10) (2, 20) (3, 30) (4, 1) (5, 40) (6, 56) ]
List, after deleting first record:
[ (5, 40) (4, 1) (3, 30) (2, 20) (1, 10) ]
List, after deleting last record:
[ (5, 40) (4, 1) (3, 30) (2, 20) ]
List, insert after key(4):
[ (5, 40) (4, 1) (7, 13) (3, 30) (2, 20) ]
List, after delete key(4):
[ (5, 40) (7, 13) (3, 30) (2, 20) ]
```

Circular Linked

Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list.

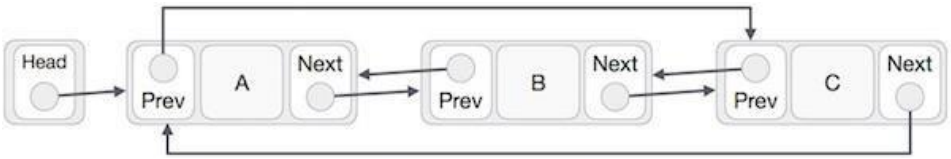
Singly Linked List as Circular

In singly linked list, the next pointer of the last node points to the first node.



Doubly Linked List as Circular

In doubly linked list, the next pointer of the last node points to the first node and the previous pointer of the first node points to the last node making the circular in both directions.



As per the above illustration, following are the important points to be considered.

- The last link's next points to the first link of the list in both cases of singly as well as doubly linked list.
- The first link's previous points to the last of the list in case of doubly linked list.

Basic Operations in Circular Linked List

Following are the important operations supported by a circular list.

- Insert − Inserts an element at the start of the list.
- Delete − Deletes an element from the start of the list.
- Display − Displays the list.

Circular Linked List - Insertion Operation

The insertion operation of a circular linked list only inserts the element at the start of the list. This differs from the usual singly and doubly linked lists as there is no particular starting and ending points in this list. The insertion is done either at the start or after a particular node (or a given position) in the list.

Algorithm

1. START

2. Check if the list is empty
3. If the list is empty, add the node and point the head to this node
4. If the list is not empty, link the existing head as the next node to the new node.
5. Make the new node as the new head.
6. END

Circular Linked List - Deletion Operation

The Deletion operation in a Circular linked list removes a certain node from the list. The deletion operation in this type of lists can be done at the beginning, or a given position, or at the ending.

Algorithm

1. START
2. If the list is empty, then the program is returned.
3. If the list is not empty, we traverse the list using a current pointer that is set to the head pointer and create another pointer previous that points to the last node.
4. Suppose the list has only one node, the node is deleted by setting the head pointer to NULL.
5. If the list has more than one node and the first node is to be deleted, the head is set to the next node and the previous is linked to the new head.
6. If the node to be deleted is the last node, link the preceding node of the last node to head node.
7. If the node is neither first nor last, remove the node by linking its preceding node to its succeeding node.
8. END

Circular Linked List - Displaying the List

The Display List operation visits every node in the list and prints them all in the output.

Algorithm

1. START
2. Walk through all the nodes of the list and print them
3. END

Circular Linked List - Complete Implementation

```
#include <iostream>          // Allows input and output
using namespace std;         // Allows use of cout without std::
// Node structure definition
```

```
struct Node {
    int key;                    // Stores the key value
    int data;                   // Stores the actual data
    Node* next;                 // Pointer to the next node
    // Constructor to initialize node values
    Node(int k, int d) : key(k), data(d), next(nullptr) {}
};
// Class to manage the circular linked list
class CircularLinkedList {
private:
    Node* head;                 // Pointer to the first node
public:
    // Constructor initializes head to null (empty list)
    CircularLinkedList() : head(nullptr) {}
    // Function to check if list is empty
    bool isEmpty() const {
        return head == nullptr;  // Returns true if head is null
    }
    // Function to count number of nodes in list
    int length() const {
        if (isEmpty())          // If list is empty
            return 0;           // Return 0 length
        int count = 1;          // Start counting from head
        Node* current = head->next;  // Start from second node
        // Traverse until we come back to head
        while (current != head) {
            count++;            // Increase count
            current = current->next; // Move to next node
        }
        return count;           // Return total count
    }
```

```
// Insert a node at the beginning
void insertFirst(int key, int data) {
    Node* newNode = new Node(key, data);  // Create new node

    if (isEmpty()) {         // If list is empty
        head = newNode;      // Head becomes new node
        head->next = head;   // Point to itself (circular link)
    } else {
        Node* temp = head;   // Temporary pointer to traverse
        // Move to last node
        while (temp->next != head) {
            temp = temp->next;
        }
        newNode->next = head;   // New node points to old head
        temp->next = newNode;   // Last node points to new node
        head = newNode;         // Head updated to new node
    }
}
// Delete first node
Node* deleteFirst() {
    if (isEmpty())           // If list empty
        return nullptr;      // Nothing to delete
    Node* temp = head;       // Store current head

    if (head->next == head) {   // If only one node exists
        head = nullptr;         // List becomes empty
    } else {
        Node* last = head;      // Pointer to find last node
        // Traverse to last node
        while (last->next != head) {
            last = last->next;
```

```
        }
        head = head->next;      // Move head to next node
        last->next = head;      // Last node points to new head
    }
    return temp;            // Return deleted node
}
// Display the list
void printList() const {
    if (isEmpty()) {         // If list is empty
        cout << "[ Empty ]\n";
        return;             // Exit function
    }
    Node* current = head;    // Start from head
    cout << "[ ";
    // Loop through circular list using do-while
    do {
        cout << "(" << current->key << "," << current->data << ") ";
        current = current->next;  // Move to next node
    } while (current != head);    // Stop when back at head
    cout << "]\n";
}
// Destructor to free memory automatically
~CircularLinkedList() {
    while (!isEmpty()) {         // While list is not empty
        Node* temp = deleteFirst();  // Delete first node
        delete temp;                // Free memory
    }
}
};
// Main function
int main() {
```

```
    CircularLinkedList list;   // Create list object
    // Insert nodes
    list.insertFirst(1, 10);
    list.insertFirst(2, 20);
    list.insertFirst(3, 30);
    list.insertFirst(4, 1);
    list.insertFirst(5, 40);
    list.insertFirst(6, 56);
    cout << "Original List:\n";
    list.printList();          // Display list
    // Delete nodes one by one
    while (!list.isEmpty()) {
        Node* temp = list.deleteFirst();  // Remove first node
        cout << "Deleted: (" << temp->key << "," << temp->data << ")\n";
        delete temp;                      // Free memory
    }
    cout << "List after deleting all items:\n";
    list.printList();          // Show empty list
    return 0;                  // End program
}
```

Output

```
Original List:
[ (6,56) (5,40) (4,1) (3,30) (2,20)  ]
 Deleted value:(6,56)
 Deleted value:(5,40)
 Deleted value:(4,1)
 Deleted value:(3,30)
 Deleted value:(2,20)
 Deleted value:(1,10)
 List after deleting all items:
[  ]
```

CHAPTER 4

STACKS AND QUEUES

STACKS

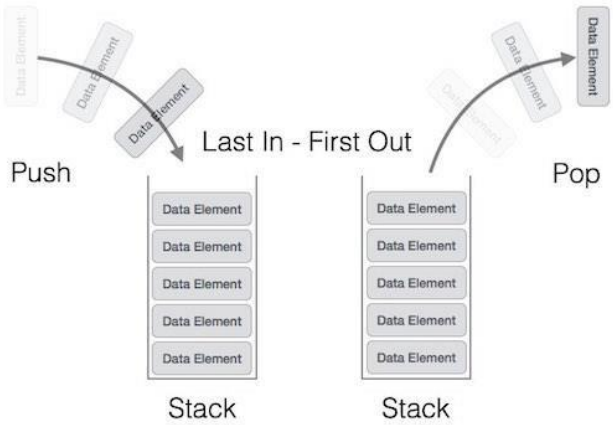
A stack is a **linear data structure** where elements are stored in the LIFO (Last In First Out) principle where the last element inserted would be the first element to be deleted. A stack is an Abstract Data Type (ADT), which is popularly used in most programming languages. It is named stack because it has the similar operations as the real-world stacks, for example − a pack of cards or a pile of plates, etc.

Stack is considered a complex data structure because it uses other data structures for implementation, such as Arrays, Linked lists, etc.

Stack Representation

A stack allows all data operations at one end only. At any given time, we can only access the top element of a stack.

The following diagram depicts a stack and its operations −



A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Either stack can be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack-using arrays, which makes it a fixed size stack implementation.

Basic Operations on Stacks

Stack operations are usually performed for initialization, usage and, de-initialization of the stack ADT.

The most fundamental operations in the stack ADT include:

1. push(),
2. pop(),
3. peek(),
4. isFull(),
5. isEmpty().

These are all built-in operations to carry out data manipulation and to check the status of the stack. Stack uses pointers that always point to the topmost element within the stack, hence called as the **top** pointer.

Stack Insertion: push()

The push() is an operation that inserts elements into the stack. The following is an algorithm that describes the push() operation in a simpler way.

Algorithm

1. Checks if the stack is full.
2. If the stack is full, produces an error and exit.
3. If the stack is not full, increments top to point next empty space.
4. Adds data element to the stack location, where top is pointing.
5. Returns success.

Example

```
#include <iostream>
using namespace std;
int MAXSIZE = 8;
int stack[8];
int top = -1;
/* Check if the stack is full*/
int isfull(){
  if(top == MAXSIZE)
    return 1;
  else
```

```
    return 0;
}
/* Function to insert into the stack */
int push(int data){
  if(!isfull()) {
    top = top + 1;
    stack[top] = data;
  } else {
    cout<<"Could not insert data, Stack is full"<<endl;
  }
  return data;
}
/* Main function */
int main(){
  int i;
  push(44);
  push(10);
  push(62);
  push(123);
  push(15);
  cout<<"Stack Elements: "<<endl;
  // print stack data
  for(i = 0; i < 8; i++) {
    cout<<stack[i]<<", ";
  }
  return 0;
}
```

Output

Stack Elements:

44 10 62 123 15 0 0 0

Stack Deletion: pop ()

The pop () is a data manipulation operation which removes elements from the stack. The following pseudo code describes the pop () operation in a simpler way.

Algorithm

1. Checks if the stack is empty.
2. If the stack is empty, produces an error and exit.
3. If the stack is not empty, accesses the data element at which top is pointing.
4. Decreases the value of top by 1.
5. Returns success.

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>
using namespace std;
int MAXSIZE = 8;
int stack[8];
int top = -1;
/* Check if the stack is empty */
int isempty(){
   if(top == -1)
      return 1;
   else
      return 0;
}
/* Check if the stack is full*/
int isfull(){
   if(top == MAXSIZE)
      return 1;
   else
      return 0;
}
/* Function to delete from the stack */
int pop(){
```

```
   int data;
   if(!isempty()) {
      data = stack[top];
      top = top - 1;
      return data;
   } else {
      cout<<"Could not retrieve data, Stack is empty."<<endl;
   }
   return data;
}
/* Function to insert into the stack */
int push(int data){
   if(!isfull()) {
      top = top + 1;
      stack[top] = data;
   } else {
      cout<<"Could not insert data, Stack is full."<<endl;
   }
   return data;
}
/* Main function */
int main(){
   int i;
   push(44);
   push(10);
   push(62);
   push(123);
   push(15);
   cout<<"Stack Elements: "<<endl;
   // print stack data
   for(i = 0; i < 8; i++) {
```

```
    cout<<stack[i]<<" ";
  }
  cout<<" \nElements popped: "<<endl;
  // print stack data
  while(!isempty()) {
    int data = pop();
    cout<<data<<" ";
  }
  return 0;
}
```

Output

Stack Elements:

44 10 62 123 15 0 0 0

Elements popped:

15 123 62 10 44

Retrieving topmost Element from Stack: peek()

The peek() is an operation retrieves the topmost element within the stack, without deleting it. This operation is used to check the status of the stack with the help of the top pointer.

Algorithm

1. START
2. Return the element at the top of the stack
3. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>
using namespace std;
int MAXSIZE = 8;
int stack[8];
int top = -1;
/* Check if the stack is full */
int isfull(){
```

```
   if(top == MAXSIZE)
      return 1;
   else
      return 0;
}
/* Function to return the topmost element in the stack */
int peek(){
   return stack[top];
}
/* Function to insert into the stack */
int push(int data){
   if(!isfull()) {
      top = top + 1;
      stack[top] = data;
   } else {
      cout<<"Could not insert data, Stack is full."<<endl;
   }
   return data;
}
/* Main function */
int main(){
   int i;
   push(44);
   push(10);
   push(62);
   push(123);
   push(15);
   push(44);
   push(10);
   push(62);
   cout<<"Stack Elements: "<<endl;
```

```
  // print stack data
  for(i = 0; i < 8; i++) {
    cout<<stack[i]<<" ";
  }
  cout<<"\nElement at top of the stack: "<<peek();
  return 0;
}
```

Output

Stack Elements:

44 10 62 123 15 0 0 0

Element at top of the stack: 15

Verifying whether the Stack is full: isFull()

The isFull() operation checks whether the stack is full. This operation is used to check the status of the stack with the help of top pointer.

Algorithm

1. START
2. If the size of the stack is equal to the top position of the stack, the stack is full. Return 1.
3. Otherwise, return 0.
4. 4. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>
int MAXSIZE = 8;
int stack[8];
int top = -1;
/* Check if the stack is full */
int isfull(){
  if(top == MAXSIZE)
    return 1;
```

```
  else
    return 0;
}
/* Main function */
int main(){
  printf("Stack full: %s\n" , isfull()?"true":"false");
  return 0;
}
```

Output

Stack full: false

Verifying whether the Stack is empty: isEmpty()

The isEmpty() operation verifies whether the stack is empty. This operation is used to check the status of the stack with the help of top pointer.

Algorithm

1. START
2. If the top value is -1, the stack is empty. Return 1.
3. Otherwise, return 0.
4. END

Example

Following are the implementations of this operation in various programming languages –

```
#include <iostream>
int MAXSIZE = 8;
int stack[8];
int top = -1;
/* Check if the stack is empty */
int isempty(){
  if(top == -1)
    return 1;
  else
    return 0;
}
```

```
/* Main function */
int main(){
   printf("Stack empty: %s\n" , isempty()?"true":"false");
   return 0;
}
```

Output

Stack empty: true

Expression Parsing in Data Structure

An expression is any word or group of words or symbols that generates a value on evaluation. Parsing expression means analyzing the expression for its words or symbols depending on a particular criterion. Expression parsing is a term used in a programming language to evaluate arithmetic and logical expressions.

The way to write arithmetic expression is known as a notation. An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression. These notations are −

- Infix Notation
- Prefix (Polish) Notation
- Postfix (Reverse-Polish) Notation

These notations are named as how they use operator in expression. We shall learn the same here in this chapter.

Infix Notation

We write expression in infix notation, e.g. a - b + c, where operators are used in-between operands. It is easy for us humans to read, write, and speak in infix notation but the same does not go well with computing devices. An algorithm to process infix notation could be difficult and costly in terms of time and space consumption.

Prefix Notation

In this notation, operator is prefixed to operands, i.e. operator is written ahead of operands. For example, +ab. This is equivalent to its infix notation a + b. Prefix notation is also known as Polish Notation.

Postfix Notation

This notation style is known as Reversed Polish Notation. In this notation style, the operator is postfixed to the operands i.e., the operator is written after the operands. For example, ab+. This is equivalent to its infix notation a + b.

The following table briefly tries to show the difference in all three notations −

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	a + b	+ a b	a b +
2	(a + b) ∗ c	∗ + a b c	a b + c ∗
3	a ∗ (b + c)	∗ a + b c	a b c + ∗
4	a / b + c / d	+ / a b / c d	a b / c d / +
5	(a + b) ∗ (c + d)	∗ + a b + c d	a b + c d + ∗
6	((a + b) ∗ c) - d	- ∗ + a b c d	a b + c ∗ d -

Parsing Expressions

As we have discussed, it is not a very efficient way to design an algorithm or program to parse infix notations. Instead, these infix notations are first converted into either postfix or prefix notations and then computed.

To parse any arithmetic expression, we need to take care of operator precedence and associativity also.

Precedence

When an operand is in between two different operators, which operator will take the operand first, is decided by the precedence of an operator over others. For example −

a + b * c ⇨ a + (b * c)

As multiplication operation has precedence over addition, b * c will be evaluated first. A table of operator precedence is provided later.

Associativity

Associativity describes the rule where operators with the same precedence appear in an expression. For example, in expression a + b − c, both + and − have the same precedence, then

which part of the expression will be evaluated first, is determined by associativity of those operators. Here, both + and − are left associative, so the expression will be evaluated as (a + b) − c.

Precedence and associativity determines the order of evaluation of an expression. Following is an operator precedence and associativity table (highest to lowest) −

Sr.No.	Operator	Precedence	Associativity
1	Exponentiation ^	Highest	Right Associative
2	Multiplication (∗) & Division (/)	Second Highest	Left Associative
3	Addition (+) & Subtraction (−)	Lowest	Left Associative

The above table shows the default behavior of operators. At any point of time in expression evaluation, the order can be altered by using parenthesis. For example −

In a + b*c, the expression part b*c will be evaluated first, with multiplication as precedence over addition. We here use parenthesis for a + b to be evaluated first, like (a + b)*c.

Postfix Evaluation Algorithm

We shall now look at the algorithm on how to evaluate postfix notation −

1. Step 1. Scan the expression from left to right
2. Step 2. If it is an operand push it to stack
3. Step 3. If it is an operator pull operand from stack and perform operation
4. Step 4. Store the output of step 3, back to stack
5. Step 5. Scan the expression until all operands are consumed
6. Step 6. Pop the stack and perform operation

QUEUES

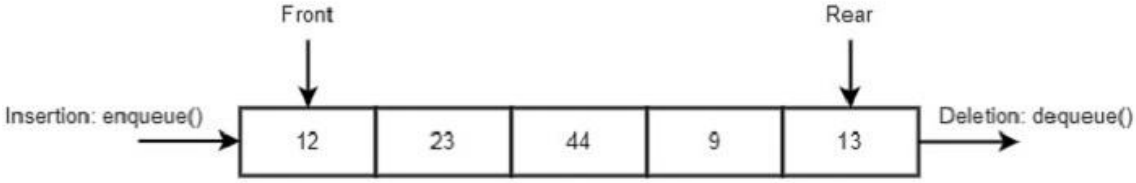
What is a Queue?

A queue is a linear data structure where elements are stored in the FIFO (First In First Out) principle where the first element inserted would be the first element to be accessed. A queue is an Abstract Data Type (ADT) similar to stack, the thing that makes queue different from stack is that a queue is open at both its ends. The data is inserted into the queue through one end and deleted from it using the other end. Queue is very frequently used in most programming languages.

A real-world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first. More real-world examples can be seen as queues at the ticket windows and bus stops.

Representation of Queues

Similar to the stack ADT, a queue ADT can also be implemented using arrays, linked lists, or pointers. As a small example in this tutorial, we implement queues using a one-dimensional array.



Queue: FIFO Operation

Basic Operations in Queue

Queue operations also include initialization of a queue, usage and permanently deleting the data from the memory.

The most fundamental operations in the queue ADT include: enqueue(), dequeue(), peek(), isFull(), isEmpty(). These are all built-in operations to carry out data manipulation and to check the status of the queue.

Queue uses two pointers − front and rear. The front pointer accesses the data from the front end (helping in enqueueing) while the rear pointer accesses data from the rear end (helping in dequeuing).

Basic key functions when dealing with queues.

Function	Purpose
q.push(x)	Inserts element at the back (enqueue)
q.pop()	Removes front element (dequeue)
q.front()	Returns front element
q.empty()	Checks if queue is empty
q.size()	Returns number of elements

Queue Insertion Operation: Enqueue()

The enqueue() is a data manipulation operation that is used to insert elements into the queue. The following algorithm describes the enqueue() operation in a simpler way.

Algorithm

1. START
2. Check if the queue is full.
3. If the queue is full, produce overflow error and exit.
4. If the queue is not full, increment rear pointer to point the next empty space.
5. Add data element to the queue location, where the rear is pointing.
6. return success.
7. END

Example

Following are the implementations of this operation in various programming languages −

```
#include <iostream>   // For input and output
#include <queue>      // For std::queue container
using namespace std;
int main() {
   queue<int> q;   // Declares a queue named q that stores integers.
   // Insert elements into the queue (enqueue)
   q.push(3);
   q.push(5);
   q.push(9);
   q.push(1);
   q.push(12);
```

```
    q.push(15);
    cout << "Queue: ";
    // Remove and print elements until queue is empty
    while(!q.empty()) { // Check if queue is not empty. Loop runs as long as the queue is NOT empty.
        cout << q.front() << " ";  // gets the element at the front of the queue and prints it
        q.pop();                   // Remove front element
    }
    return 0;   // Indicate successful execution
}
```

Output

Queue: 3 5 9 1 12 15

Queue Deletion Operation: dequeue()

The dequeue() is a data manipulation operation that is used to remove elements from the stack. The following algorithm describes the dequeue() operation in a simpler way.

Algorithm

1. START
2. Check if the queue is empty.
3. If the queue is empty, produce underflow error and exit.
4. If the queue is not empty, access the data where front is pointing.
5. Increment front pointer to point to the next available data element.
6. Return success.
7. END

Example

```
#include <iostream>     // Allows input and output using cout and endl
#include <queue>        // Allows use of the queue container (std::queue)
using namespace std;    // Allows us to write queue and cout without std::
int main() {            // Program execution starts here
    queue<int> q;       // Create a queue that stores integers
    // Insert 6 items into the queue (enqueue operation)
    q.push(3);          // Add 3 at the back of the queue
```

```
    q.push(5);          // Add 5 at the back
    q.push(9);          // Add 9 at the back
    q.push(1);          // Add 1 at the back
    q.push(12);         // Add 12 at the back
    q.push(15);         // Add 15 at the back
    cout << "Queue: ";  // Print label
    // Create a copy of the queue so we can display elements
    // without removing them from the original queue
    queue<int> temp = q;
    // Loop while copied queue is not empty
    while(!temp.empty()) {      // Check if temp queue still has elements
        cout << temp.front() << " ";  // Print front element
        temp.pop(); // Remove front element from temp
    }
    // Remove one item from original queue
    int num = q.front();   // Get the front element of original queue
    q.pop();               // Remove the front element
    cout << "\nElement removed: " << num << endl;  // Print removed element
    cout << "Updated Queue: ";   // Print label for updated queue
    // Display remaining elements in original queue
    while(!q.empty()) {          // While queue still has elements
        cout << q.front() << " "; // Print front element
        q.pop();                  // Remove front element
    }

    return 0;   // Indicate successful program termination
}
```

Output

Queue: 3 5 9 1 12 15

Element removed: 3

Updated Queue: 5 9 1 12 15

Queue - The peek() Operation

The peek() is an operation which is used to retrieve the frontmost element in the queue, without deleting it. This operation is used to check the status of the queue with the help of the pointer.

Algorithm

1. START
2. Return the element at the front of the queue
3. END

Example

```
#include <iostream>    // Allows input and output using cout and endl
#include <queue>       // Allows use of the queue container (std::queue)
using namespace std;   // Allows us to use queue and cout without writing std::
int main() {        // Main function - program execution starts here
   queue<int> q;    // Declare a queue named 'q' that stores integers
   // Insert 6 items into the queue (enqueue operation)
   q.push(3);        // Add 3 to the back of the queue
   q.push(5);        // Add 5 to the back
   q.push(9);        // Add 9 to the back
   q.push(1);        // Add 1 to the back
   q.push(12);        // Add 12 to the back
   q.push(15);        // Add 15 to the back
   cout << "Queue: "; // Print label before displaying elements
   // Create a copy of the queue so we can display elements
   // without removing them from the original queue
   queue<int> temp = q;
   // Loop through the copied queue
   while (!temp.empty()) {      // Continue while temp is not empty
      cout << temp.front() << " "; // Print front element
      temp.pop();                // Remove front element from temp
   }
   // Display the element at the front (peek operation)
   cout << "\nElement at front: " << q.front() << endl;
```

```
    // q.front() returns the front element without removing it

    return 0;   // Indicate successful program termination
}
```

Output

Queue: 3 5 9 1 12 15

Element at front: 3

Queue - The isFull() Operation

The isFull() operation verifies whether the stack is full.

Algorithm

1. START
2. If the count of queue elements equals the queue size, return true
3. Otherwise, return false
4. END

Example

Following are the implementations of this operation in various programming languages −

```
#include <iostream>    // Allows input and output using cout and endl
#include <queue>       // Allows use of the queue container (std::queue)
using namespace std;   // Allows us to use queue and cout without writing std::
int main() {           // Program execution starts here
    queue<int> q;      // Declare a queue that stores integers
    // Insert 6 items into the queue (enqueue operation)
    q.push(3);         // Add 3 to the back of the queue
    q.push(5);         // Add 5 to the back
    q.push(9);         // Add 9 to the back
    q.push(1);         // Add 1 to the back
    q.push(12);        // Add 12 to the back
    q.push(15);        // Add 15 to the back
    cout << "Queue: "; // Print label before displaying elements
    // Create a temporary copy so we can display elements
    // without removing them from the original queue
    queue<int> temp = q;
```

```
    // Loop through the copied queue
    while (!temp.empty()) {      // Continue while temp queue is not empty
        cout << temp.front() << " "; // Print the front element
        temp.pop();                  // Remove front element from temp
    }
    cout << endl;   // Move to the next line
    // Check if queue is full
    // Since std::queue does not have a fixed MAX size,
    // we manually compare size with 6 (original MAX)
    if (q.size() == 6) {       // If queue contains 6 elements
        cout << "Queue is full!" << endl;  // Print message
    }

    return 0;   // Indicate successful program termination
}
```

Output

Queue: 3 5 9 1 12 15

Queue is full!

Queue - The isEmpty() operation

The isEmpty() operation verifies whether the stack is empty. This operation is used to check the status of the stack with the help of top pointer.

Algorithm

1. START
2. If the count of queue elements equals zero, return true
3. Otherwise, return false
4. END

Example

Following are the implementations of this operation in various programming languages −

```
#include <iostream>    // Allows use of input and output (cout, endl)
#include <queue>       // Allows use of std::queue container
using namespace std;   // Allows us to write queue and cout without std::
```

```
int main() {          // Program execution starts here
    queue<int> q;      // Declare an empty queue of integers
    cout << "Queue: "; // Print label before displaying elements
    // Create a temporary copy of the queue
    // (Currently the queue is empty)
    queue<int> temp = q;

    // Loop to display elements (will not execute because queue is empty)
    while (!temp.empty()) {      // Check if temp queue is not empty
        cout << temp.front() << " "; // Print front element
        temp.pop();                  // Remove front element from temp
    }
    cout << endl;   // Move to next line
    // Check if queue is empty
    if (q.empty()) {  // empty() returns true if queue has no elements
        cout << "Queue is Empty!" << endl;  // Print message
    }
    return 0;   // End of program (successful execution)
}
```

Output

Queue: 0 0 0 0 0 0

Queue is Empty!

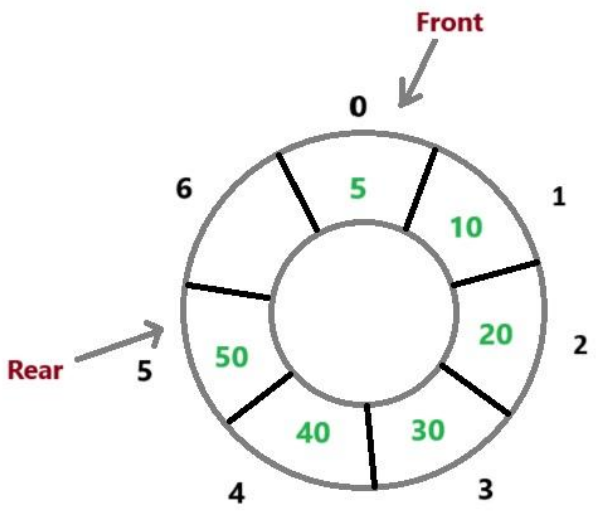
Circular Queue

A **circular queue** is a type of queue in which the last position is connected back to the first position to make a circle. It is also known as a **Ring Buffer.** In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. However, using a circular queue, we can use the space to insert elements.

Circular queue is a linear data structure that follows the FIFO mechanism. The circular queue is a more efficient way to implement a queue in a fixed size array. In a circular queue, the last element points to the first element making a circular link.

Representation of Circular Queue

Following is the representation of a circular queue, where the front is the index of the first element, and the rear is the index of the last element.



Operations on Circular Queue

There are mainly four operations that can be performed on a circular queue:

1. Enqueue: Insert an element into the circular queue.
2. Dequeue: Delete an element from the circular queue.
3. Front: Get the front element from the circular queue.
4. Rear: Get the last element from the circular queue.

Enqueue Operation on Circular Queue

Enqueue operation is used for inserting an element into the circular queue. The steps to perform the enqueue operation are as follows:

Algorithm for Enqueue Operation

Following are the steps to perform the enqueue operation on a circular queue:

1. Initialize an array or any data structure to store the elements of the circular queue.
2. Initialize two variables front and rear.
3. Check if the circular queue is full.
4. If it is not full, and if the front is -1, set the front to 0.
5. Increment the rear by 1 and store it in the rear index.
6. Update the rear index using rear = (rear + 1) % SIZE.

Dequeue Operation on Circular Queue

Dequeue operation is used for deleting an element from the circular queue. The steps to perform the dequeue operation are as follows:

Algorithm for Dequeue Operation

Following are the steps to perform the dequeue operation on a circular queue:

1. Check if the circular queue is empty.
2. If the queue is not empty, store the element at the front index.
3. If the front and rear are equal, set the front and rear to -1.
4. Else, increase the front index by 1 using the formula front = (front + 1) % SIZE.
5. At last print the deleted element.

Front Operation on Circular Queue

Front operation is used to get the front element from the circular queue. The steps to perform the front operation are as follows:

Algorithm for Front Operation

1. Check if the circular queue is empty.
2. If not empty, print the front index element/

Rear Operation on Circular Queue

Rear operation is used to get the last element from the circular queue. The steps to perform the rear operation are as follows:

Algorithm for Rear Operation

1. Check if the circular queue is empty..
2. If it is not empty, print the element at the rear index.

Full Implementation of Circular Queues using Arrays

```
#include <iostream>
#include <queue>
using namespace std;
// Check if queue is empty
bool isEmpty(queue<int> &q) {
   return q.empty();
}
// Insert multiple elements into queue
void enQueue(queue<int> &q) {
```

```
    int n, value;
    cout << "How many elements do you want to insert? ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> value;
        q.push(value);                // Insert into queue
        cout << "Inserted " << value << endl;
    }
}
//fetch the front element
void displayFront(queue<int> &q){
    if(!isEmpty(q)){
        cout<<"The first elemet is: "<<q.front();
    }else {
    cout<<"Queue is empty";}
}
//Fetch the rear Element
void displayRear(queue<int> &q){
    if(!isEmpty(q)){
        cout<<"The first elemet is: "<<q.back();
    }else {
    cout<<"Queue is empty";}
}
// Delete element from queue
void deQueue(queue<int> &q) {
    if (isEmpty(q)) {
        cout << "Queue is empty" << endl;
    } else {
        int element = q.front();
        q.pop();
```

```
        cout << "Deleted element " << element << endl;
    }
}
// Display queue elements
void display(queue<int> q) {   // Pass by value
    if (q.empty()) {
        cout << "Empty Queue" << endl;
        return;
    }
    cout << "Queue Items: ";
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;
}
int main() {
    queue<int> q;
    int choice;
    do {
        cout << "\n1. Insert Multiple Elements\n";
        cout << "2. Get First Element\n";
        cout << "3. Get Last Element\n";
        cout << "4. Delete\n";
        cout << "5. Display\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                enQueue(q);   // Insert many elements
```

```
            break;
        case 2:
            displayFront(q);
            break;
        case 3:
            displayRear(q);
            break;
        case 4:
            deQueue(q);
            break;
        case 5:
            display(q);
            break;
        case 6:
            cout << "Exiting program..." << endl;
            break;
        default:
            cout << "Invalid choice!" << endl;
    }
  } while (choice != 6);
  return 0;
}
```

Full Implementation of Circular Queues using linked list

```
#include <iostream>
using namespace std;
// Node structure for linked list
struct Node {
  int data;     // Stores queue element
  Node* next;    // Pointer to next node
};
// Front and rear pointers
```

```
Node* front = NULL;
Node* rear = NULL;
// Check if queue is empty
bool isEmpty() {
    return (front == NULL);

// Insert multiple elements into queue
void enQueue() {
    int n, value;
    cout << "How many elements do you want to insert? ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> value;
        // Create new node
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = NULL;
        // If queue is empty
        if (rear == NULL) {
            front = rear = newNode;
        }
        else {
            rear->next = newNode;  // Link new node at rear
            rear = newNode;        // Move rear pointer
        }
        cout << "Inserted " << value << endl;
    }
}
// Get first element
void displayFront() {
```

```
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
    }
    else {
        cout << "First Element: " << front->data << endl;
    }
}
// Get last element
void displayRear() {
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
    }
    else {
        cout << "Last Element: " << rear->data << endl;
    }
}
// Delete element (FIFO)
void deQueue() {
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
    }
    else {
        Node* temp = front;  // Store front node
        cout << "Deleted element " << front->data << endl;
        front = front->next;  // Move front pointer
        // If queue becomes empty after deletion
        if (front == NULL) {
            rear = NULL;
        }
        delete temp;  // Free memory
    }
```

```
}
// Display all elements
void display() {
    if (isEmpty()) {
        cout << "Empty Queue" << endl;
        return;
    }
    Node* temp = front;
    cout << "Queue Items: ";
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;  // Move to next node
    }
    cout << endl;
}
// Main function
int main() {
    int choice;

    do {
        cout << "\n1. Insert Multiple Elements\n";
        cout << "2. Get First Element\n";
        cout << "3. Get Last Element\n";
        cout << "4. Delete\n";
        cout << "5. Display\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                enQueue();
```

```
            break;
        case 2:
            displayFront();
            break;
        case 3:
            displayRear();
            break;
        case 4:
            deQueue();
            break;
        case 5:
            display();
            break;
        case 6:
            cout << "Exiting program..." << endl;
            break;
        default:
            cout << "Invalid choice!" << endl;
    }
  } while (choice != 6);
  return 0;
}
```

Time Complexity of Circular Queue

- Following are the time complexities of the circular queue:
- **Enqueue Operation:** The time complexity of the enQueue operation is O(1) as we can insert an element at the rear end of the queue in constant time.
- **Dequeue Operation:** The time complexity of the deQueue operation is O(1) as we can delete an element from the front end of the queue in constant time.
- **Front Operation:** The time complexity of the front operation is O(1) as we can get the front element of the queue in constant time.

- **Rear Operation:** The time complexity of the rear operation is O(1) as we can get the rear element of the queue in constant time.

Applications of Circular Queue

- **Memory Management:** Circular queues are used in memory management to manage the memory efficiently. It is used to allocate memory to the processes when they are in need of memory.
- **Buffer Management:** These queues are also useful in buffer management. Consider a situation where data is produced at a faster rate than it is consumed. In such cases, a circular queue is used to store the data temporarily.
- **Operating System:** Suppose your system has a lot of processes to execute. In such cases, for the better management of processes, the operating system uses a circular queue to allocate the CPU to the processes.
- **Traffic Management:** Traffic signals are controlled by the circular queue. Let's say there are three signals, red, yellow, and green. The signals are in a circular queue. When the green signal is on, the next signal will be yellow, and then red. This process will continue in a circular manner.
- **Multimedia Player:** When we play songs in a multimedia player, the songs are played in a circular manner. Once the last song is played, the first song will be played next. This is done using a circular queue.

In general, when certain tasks are repeated in a circular manner, a circular queue is used to manage the tasks efficiently. Also, when the data is produced at a faster rate than it is consumed, a circular queue is used to store the data temporarily.

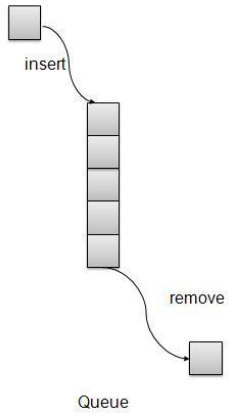
Priority Queue

Priority Queue is more specialized data structure than Queue. Like ordinary queue, priority queue has same method but with a major difference. In Priority, queue items are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa. Therefore, we assign priority to item based on its key value, the lower the value, higher the priority.

Basic Operations

- Insert / enqueue − add an item to the rear of the queue.
- Remove / dequeue − remove an item from the front of the queue.

Priority Queue Representation

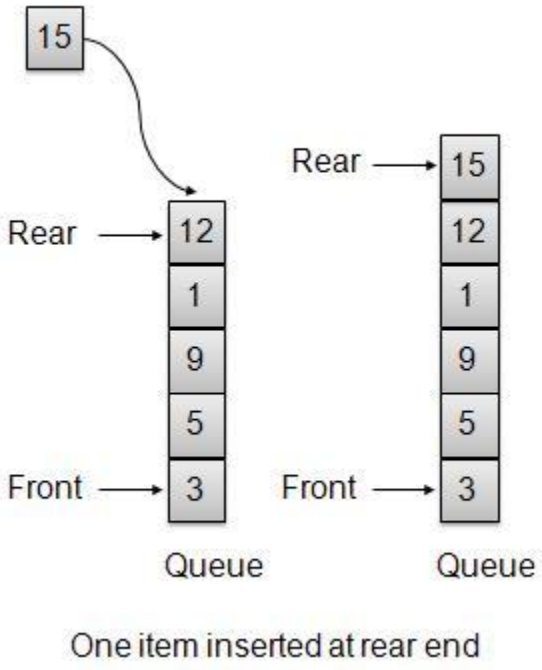


There is few more operations supported by queue which are following.

- Peek − get the element at front of the queue.
- isFull − check if queue is full.
- isEmpty − check if queue is empty.

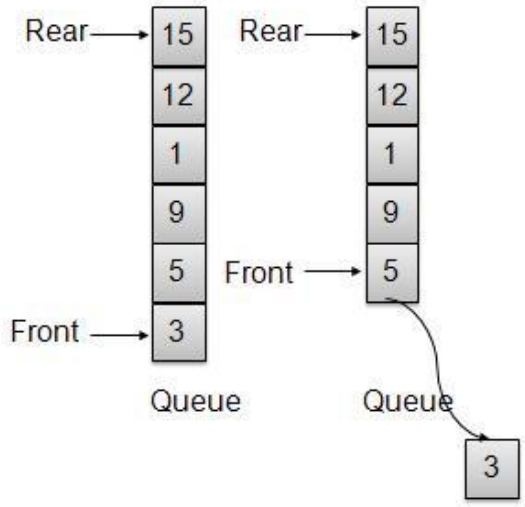
Insert / Enqueue Operation

Whenever an element is inserted into queue, priority queue inserts the item according to its order. Here we're assuming that data with high value has low priority.



Remove / Dequeue Operation

Whenever an element is to be removed from queue, queue get the element using item count. Once element is removed. Item count is reduced by one.



One Item removed from front

Deque is a hybrid data structure that combines the features of a stack and a queue. It allows us to insert and delete elements from both ends of the queue. The name Deque is an abbreviation of Double-Ended Queue.

Imagine an event where you have two gates to enter and exit a place. People are entering from the front gate and some are entering from the side gate. Now, when people are leaving, they are leaving from the front gate and some sneak from the side gate. Now, we need to manage flow of people from both ends. This is where Deque comes into play.

Operations on Deque

Following are the major operations on Deque −

- push_front(x): Insert element x at the front of the deque.
- push_back(x): Insert element x at the back of the deque.
- pop_front(): Remove the element from the front of the deque.
- pop_back(): Remove the element from the back of the deque.
- peek_front(): Get the element from the front of the deque.
- peek_back(): Get the element from the back of the deque.
- size(): Get the number of elements in the deque.
- isEmpty(): Check if the deque is empty.

Implementation of Deque

For this, we need to maintain two pointers, front and rear, to keep track of the front and back of the deque. We also need to define the size of the deque.

The push_front(x) Operation on Deque

When we insert an element at the front of the deque, we need to shift all the elements to the right by one position. We will increment the front pointer by one and insert the element at the front of **the deque.**

Algorithm for push_front(x)

Following are the steps to insert an element at the front of the deque −

1. Check if the deque is full. If it is full, return an error message.
2. Increment the front pointer by one.
3. Insert the element at the front position.
4. Increment the size of the deque.

implementation

```
#include <iostream>
using namespace std;
class Deque {
private:
   static constexpr int SIZE = 5;  // Fixed size
   int arr[SIZE];                  // Array to store elements
   int front;                      // Front index
   int rear;                       // Rear index
public:
   // Constructor initializes empty deque
   Deque() : front(-1), rear(-1) {}
   // Check if empty
   bool isEmpty() const {
      return front == -1;
   }
   // Check if full
   bool isFull() const {
```

```
    return front == (rear + 1) % SIZE;
}
// Insert at front
void push_front(int element) {
    if (isFull()) {
        cout << "Deque is full" << endl;
        return;
    }
    if (isEmpty()) {
        front = rear = 0;  // First insertion
    } else {
        front = (front - 1 + SIZE) % SIZE;  // Circular decrement
    }
    arr[front] = element;
    cout << "Inserted -> " << element << endl;
}
// Display elements
void display() const {
    if (isEmpty()) {
        cout << "Empty Deque" << endl;
        return;
    }
    cout << "Elements -> ";
    int i = front;
    while (true) {
        cout << arr[i] << " ";
        if (i == rear) break;
        i = (i + 1) % SIZE;  // Circular increment
    }
    cout << endl;
}
```

```
};
int main() {
   Deque dq;
   dq.push_front(1);
   dq.push_front(2);
   dq.push_front(3);
   dq.push_front(4);
   dq.push_front(5);
   dq.display();
   return 0;
}
```

Output

The output produced is as follows −

Inserted -> 1

Inserted -> 2

Inserted -> 3

Inserted -> 4

Inserted -> 5

Elements -> 5 4 3 2 1

push_back(x) Operation

This operation is used for inserting an element to the back of the deque. When we insert an element at the back of the deque, we need to increment the rear pointer by one and insert the element at the back of the deque.

Algorithm for push_back(x)

Following are the steps to insert an element at the back of the deque −

1. Check if the deque is full.
2. Increment the rear pointer by one.
3. Insert the element at the rear position.
4. Increment the size of the deque.

Implementation

```
#include <iostream>
```

```
using namespace std;
class Deque {
private:
    static constexpr int SIZE = 5;  // Constant size
    int arr[SIZE];                  // Array to store elements
    int front;                      // Front index
    int rear;                       // Rear index
public:
    // Constructor initializes deque as empty
    Deque() : front(-1), rear(-1) {}

    // Check if deque is empty
    bool isEmpty() const {
        return front == -1;
    }
    // Check if deque is full
    bool isFull() const {
        return (front == (rear + 1) % SIZE);
    }
    // Insert element at the back
    void push_back(int element) {
        if (isFull()) {
            cout << "Deque is full" << endl;
            return;
        }
        if (isEmpty()) {
            front = rear = 0;  // First insertion
        } else {
            rear = (rear + 1) % SIZE;  // Circular increment
        }
        arr[rear] = element;
```

```
        cout << "Inserted -> " << element << endl;
    }
    // Display deque elements
    void display() const {
        if (isEmpty()) {
            cout << "Empty Deque" << endl;
            return;
        }
        cout << "Elements -> ";
        int i = front;

        while (true) {
            cout << arr[i] << " ";
            if (i == rear) break;
            i = (i + 1) % SIZE;
        }
        cout << endl;
    }
};
int main() {
    Deque dq;
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);
    dq.push_back(5);
    dq.display();
    return 0;
}
```

Output

The output produced is as follows –

Inserted -> 1

Inserted -> 2

Inserted -> 3

Inserted -> 4

Inserted -> 5

Elements -> 1 2 3 4 5

The pop_front() and pop_back() Operations on Deque

These operation is done when we need to remove elements from front or back. When we remove an element from the front of the deque, we need to increment the front pointer by one.

Similarly, when we remove an element from the back of the deque, we need to decrement the rear pointer by one.

Algorithm for pop_front() and pop_back()

Following are the steps to remove an element from the front or back of the deque −

1. Check if the deque is empty.
2. Remove the element from the front or back of the deque.
3. Increment or decrement the front or rear pointer.

Decrement the size of the deque.

Implementation

```
#include <iostream>
#include <array>
using namespace std;   // Allows use of cout, array, etc. without std::
class Deque {
private:
   static constexpr int SIZE = 5;  // Constant size of deque
   array<int, SIZE> arr{};         // Fixed-size array container
   int front = -1;                 // Front index
   int rear  = -1;                 // Rear index
public:
   // Check if deque is empty
   bool isEmpty() const {
      return front == -1;
```

```
}
// Check if deque is full
bool isFull() const {
    return front == (rear + 1) % SIZE;
}
// Insert element at the back
void push_back(int element) {
    if (isFull()) {
        cout << "Deque is full\n";
        return;
    }
    if (isEmpty()) {
        front = rear = 0;        // First element
    } else {
        rear = (rear + 1) % SIZE; // Circular increment
    }
    arr[rear] = element;
    cout << "Inserted -> " << element << endl;
}
// Remove element from the front
void pop_front() {
    if (isEmpty()) {
        cout << "Deque is empty\n";
        return;
    }
    cout << "Deleted from front -> " << arr[front] << endl;
    if (front == rear) {  // Only one element left
        front = rear = -1;
    } else {
        front = (front + 1) % SIZE;
    }
```

```
    }
    // Remove element from the back
    void pop_back() {
        if (isEmpty()) {
            cout << "Deque is empty\n";
            return;
        }
        cout << "Deleted from back -> " << arr[rear] << endl;
        if (front == rear) {  // Only one element
            front = rear = -1;
        } else {
            rear = (rear - 1 + SIZE) % SIZE;
        }
    }
    // Display all elements
    void display() const {
        if (isEmpty()) {
            cout << "Empty Deque\n";
            return;
        }
        cout << "Elements -> ";
        int i = front;
        while (true) {
            cout << arr[i] << " ";
            if (i == rear) break;
            i = (i + 1) % SIZE;
        }
        cout << endl;
    }
};
int main() {
```

```
    Deque dq;
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);
    dq.push_back(5);
    dq.pop_front();
    dq.pop_back();
    dq.display();
    return 0;
}
```

Output

The output produced is as follows −

Inserted -> 1

Inserted -> 2

Inserted -> 3

Inserted -> 4

Inserted -> 5

Deleted from front -> 1

Deleted from back -> 5

Elements -> 2 3 4

The peek_front() and peek_back() Operations on Deque

When we want to get the element from the front or back of the deque, we can use the peek_front() and peek_back() operations.

Algorithm for peek_front() and peek_back()

Following are the steps to get the element from the front or back of the deque −

1. Check if the deque is empty.
2. If not empty, return the element from the front or back of the deque.

Implementation

```
#include <iostream>
using namespace std;
```

```
class Deque {
private:
    static const int SIZE = 5;  // Maximum size of deque
    int arr[SIZE];              // Array to store elements
    int front;                  // Points to front element
    int rear;                   // Points to rear element
public:
    // Constructor
    Deque() : front(-1), rear(-1) {}
    // Check if deque is full
    bool isFull() const {
        return ((front == 0 && rear == SIZE - 1) ||
                (front == rear + 1));
    }
    // Check if deque is empty
    bool isEmpty() const {
        return (front == -1);
    }
    // Insert element at the back
    void push_back(int element) {
        if (isFull()) {
            cout << "Deque is full\n";
            return;
        }
        if (isEmpty()) {            // If deque is empty
            front = rear = 0;
        }
        else if (rear == SIZE - 1) {  // Wrap around
            rear = 0;
        }
        else {
```

```
            rear++;
        }
        arr[rear] = element;
        cout << "Inserted -> " << element << endl;
    }
    // Get element at front
    int peek_front() const {
        if (isEmpty()) {
            cout << "Deque is empty\n";
            return -1;
        }
        return arr[front];
    }
    // Get element at back
    int peek_back() const {
        if (isEmpty()) {
            cout << "Deque is empty\n";
            return -1;
        }
        return arr[rear];
    }
};
int main() {
    Deque dq;
    dq.push_back(1);
    dq.push_back(2);
    dq.push_back(3);
    dq.push_back(4);
    dq.push_back(5);
    cout << "\nElement at front: " << dq.peek_front() << endl;
    cout << "Element at back: " << dq.peek_back() << endl;
```

```
    return 0;
}
```

Output

The output obtained is as follows −

Inserted -> 1

Inserted -> 2

Inserted -> 3

Inserted -> 4

Inserted -> 5

Element at front: 1

Element at back: 5

Time Complexity of Deque Operations

The time complexity of the deque operations is as follows −

- push_front(x) − O(1)
- push_back(x) − O(1)
- pop_front() − O(1)
- pop_back() − O(1)
- peek_front() − O(1)
- peek_back() − O(1)

Thus, the deque operations have a time complexity of O(1).

Applications of Deque

Some of the applications of deque are as follows −

- Deque is used for undo operation in text editors.
- It is also used in implementation of the sliding window algorithm.
- Deque is used in implementing the data structures like double-ended priority queue and double-ended stack.

In summary, we use deque when we need to perform insertion and deletion operations at both ends of the data structure.

CHAPTER 5

SEARCHING TECHNIQUES

Searching is a process of finding a particular record, which can be a single element or a small chunk, within a huge amount of data. The data can be in various forms: arrays, linked lists, trees, heaps, etc. With the increasing amount of data nowadays, there are multiple techniques to perform the searching operation.

Searching Algorithms in Data Structures

Various searching techniques can be applied on the data structures to retrieve certain data. A search operation is said to be successful only if it returns the desired element or data; otherwise, the searching method is unsuccessful.

There are two categories these searching techniques fall into. They are −

- Sequential Searching
- Interval Searching

Sequential Searching

As the name suggests, the sequential searching operation traverses through each element of the data sequentially to look for the desired data. The data need not be in a sorted manner for this type of search.

Example − Linear Search

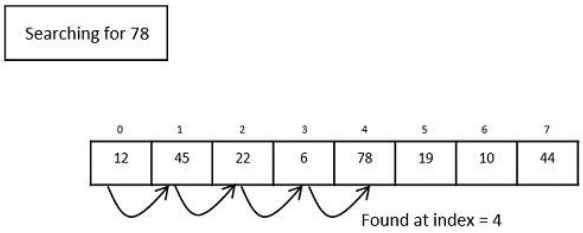


Fig. 1: Linear Search Operation

Interval Searching

Unlike sequential searching, the interval searching operation requires the data to be in a sorted manner. This method usually searches the data in intervals; it could be done by either dividing the data into multiple sub-parts or jumping through the indices to search for an element.

Example − Binary Search, Jump Search etc.

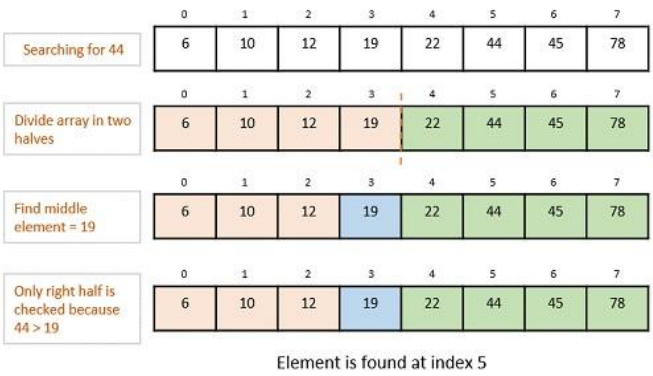


Fig. 2: Binary Search Operation

Evaluating Searching Algorithms

Usually, not all searching techniques are suitable for all types of data structures. In some cases, a sequential search is preferable while in other cases interval searching is preferable. Evaluation of these searching techniques is done by checking the running time taken by each searching method on a particular input.

To explain briefly, there are three different cases of time complexity in which a program can run. They are −

- Best Case
- Average Case
- Worst Case

We mostly concentrate on the only best-case and worst-case time complexities, as the average case is difficult to compute. In addition, since the running time is based on the amount of input given to the program, the worst-case time complexity best describes the performance of any algorithm.

For instance, the best case time complexity of a linear search is O(1) where the desired element is found in the first iteration; whereas the worst case time complexity is O(n) when the program traverses through all the elements and still does not find an element. This is labelled as an unsuccessful search. Therefore, the actual time complexity of a linear search is seen as O(n), where n is the number of elements present in the input data structure.

Many types of searching methods are used to search for data entries in various data structures. Some of them include −

- Linear Search

- Binary Search
- Interpolation Search
- Jump Search
- Hash Table
- Exponential Search
- Sublist search
- Fibonacci Search
- Ubiquitous Binary Search

Linear Search Algorithm

Linear search is a type of sequential searching algorithm. In this method, every element within the input array is traversed and compared with the key element to be found. If a match is found in the array the search is said to be successful; if there is no match found the search is said to be unsuccessful and gives the worst-case time complexity.

For instance, in the given animated diagram, we are searching for an element 33. Therefore, the linear search method searches for it sequentially from the very first element until it finds a match. This returns a successful search.

Linear Search Algorithm

The algorithm for linear search is relatively simple. The procedure starts at the very first index of the input array to be searched.

Step 1 − Start from the 0th index of the input array, compare the key value with the value present in the 0th index.

Step 2 − If the value matches with the key, return the position at which the value was found.

Step 3 − If the value does not match with the key, compare the next element in the array.

Step 4 − Repeat Step 3 until there is a match found. Return the position at which the match was found.

Step 5 − If it is an unsuccessful search, print that the element is not present in the array and exit the program.

Pseudocode

```
procedure linear_search (list, value)
   for each item in the list
      if match item == value
```

```
        return the item's location
      end if
    end for
end procedure
```

Analysis

Linear search traverses through every element sequentially therefore, the best case is when the element is found in the very first iteration. The best-case time complexity would be O(1).

However, the worst case of the linear search method would be an unsuccessful search that does not find the key value in the array, it performs n iterations. Therefore, the worst-case time complexity of the linear search algorithm would be O(n).

Example

Let us look at the step-by-step searching of the key element (say 47) in an array using the linear search method.

0	1	2	3	4	5	6	7
34	10	66	27	47	8	55	78

Step 1

The linear search starts from the 0th index. Compare the key element with the value in the 0th index, 34.

0	1	2	3	4	5	6	7
34	10	66	27	47	8	55	78
= 47							

However, 47 34. So it moves to the next element.

Step 2

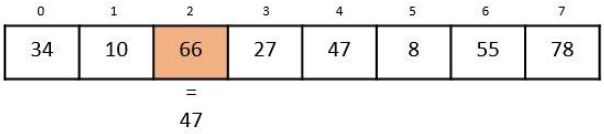
Now, the key is compared with value in the 1st index of the array.

0	1	2	3	4	5	6	7
34	10	66	27	47	8	55	78
	= 47						

Still, 47 10, making the algorithm move for another iteration.

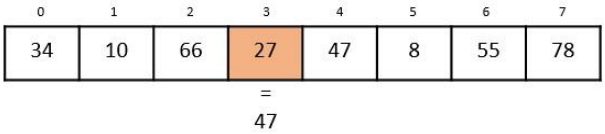
Step 3

The next element 66 is compared with 47. They are both not a match so the algorithm compares the further elements.



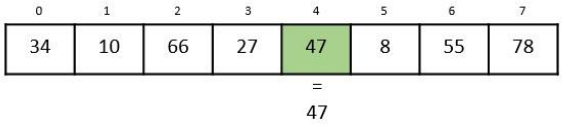
Step 4

Now the element in 3rd index, 27, is compared with the key value, 47. They are not equal so the algorithm is pushed forward to check the next element.



Step 5

Comparing the element in the 4th index of the array, 47, to the key 47. It is figured that both the elements match. Now, the position in which 47 is present, i.e., 4 is returned.



The output achieved is Element found at 4 th index.

Implementation

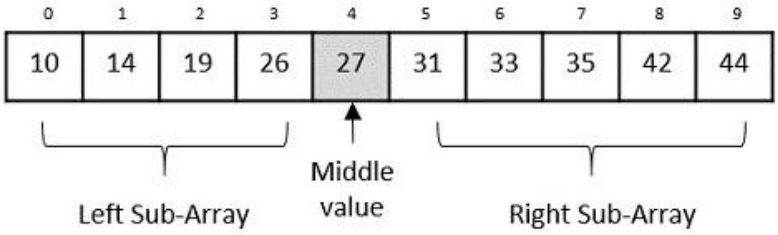
```
 #include <iostream>
using namespace std;
// Function to perform linear search
int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {     // Traverse array
        if (arr[i] == target) {          // If element found
```

```
            return i;                    // Return index
        }
    }
    return -1;   // Return -1 if not found
}
int main() {
    int arr[] = {10, 25, 5, 40, 15};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target;
    cout << "Enter number to search: ";
    cin >> target;
    int result = linearSearch(arr, size, target);
    if (result != -1)
        cout << "Element found at index " << result << endl;
    else
        cout << "Element not found" << endl;
    return 0;
}
```

Binary Search Algorithm

Binary search is a fast search algorithm with run-time complexity of (log n). This search algorithm works on the principle of divide and conquer, since it divides the array into half before searching. For this algorithm to work properly, the data collection should be in the sorted form.

Binary search looks for a particular key value by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. But if the middle item has a value greater than the key value, the right sub-array of the middle item is searched. Otherwise, the left sub-array is searched. This process continues recursively until the size of a subarray reduces to zero.



Binary Search Algorithm

Binary Search algorithm is an interval searching method that performs the searching in intervals only. The input taken by the binary search algorithm must always be in a sorted array since it divides the array into subarrays based on the greater or lower values. The algorithm follows the procedure below −

Step 1 − Select the middle item in the array and compare it with the key value to be searched. If it is matched, return the position of the median.

Step 2 − If it does not match the key value, check if the key value is either greater than or less than the median value.

Step 3 − If the key is greater, perform the search in the right sub-array; but if the key is lower than the median value, perform the search in the left sub-array.

Step 4 − Repeat Steps 1, 2 and 3 iteratively, until the size of sub-array becomes 1.

Step 5 − If the key value does not exist in the array, then the algorithm returns an unsuccessful search.

Pseudocode

The pseudocode of binary search algorithms should look like this −

```
Procedure binary_search
  A ← sorted array
  n ← size of array
  x ← value to be searched
  Set lowerBound = 1
  Set upperBound = n
  while x not found
    if upperBound < lowerBound
      EXIT: x does not exists.
```

```
    set midPoint = lowerBound + ( upperBound - lowerBound ) / 2
    if A[midPoint] < x
      set lowerBound = midPoint + 1
    if A[midPoint] > x
      set upperBound = midPoint - 1
    if A[midPoint] = x
      EXIT: x found at location midPoint
  end while
end procedure
```

Analysis

Since the binary search algorithm performs searching iteratively, calculating the time complexity is not as easy as the linear search algorithm.

The input array is searched iteratively by dividing into multiple sub-arrays after every unsuccessful iteration. Therefore, the recurrence relation formed would be of a dividing function.

To explain it in simpler terms,

During the first iteration, the element is searched in the entire array. Therefore, length of the array = n.

In the second iteration, only half of the original array is searched. Hence, length of the array = n/2.

In the third iteration, half of the previous sub-array is searched. Here, length of the array will be = n/4.

Similarly, in the ith iteration, the length of the array will become n/2i

To achieve a successful search, after the last iteration the length of array must be 1. Hence,

n/2i = 1

That gives us −

n = 2i

Applying log on both sides,

log n = log 2i

log n = i. log 2

i = log n

The time complexity of the binary search algorithm is O(log n)

Example

For a binary search to work, it is mandatory for the target array to be sorted. We shall learn the process of binary search with a pictorial example. The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44

First, we shall determine half of the array by using this formula −

mid = low + (high - low) / 2

Here it is, 0 + (9 - 0) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44

Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44

We change our low to mid + 1 and find the new mid value again.

low = mid + 1

mid = low + (high - low) / 2

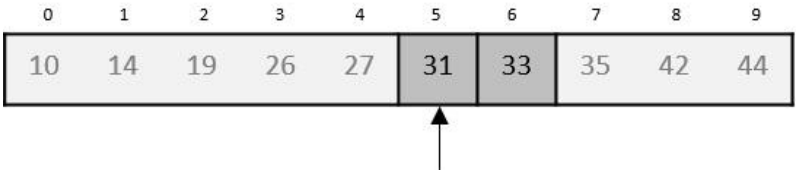
Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44

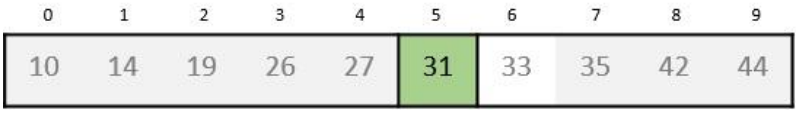
The value stored at location 7 is not a match, rather it is less than what we are looking for. So, the value must be in the lower part from this location.

0	1	2	3	4	5	6	7	8	9
10	14	19	26	27	31	33	35	42	44

Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.



We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Implementation

Binary search is a fast search algorithm with run-time complexity of (log n). This search algorithm works on the principle of divide and conquer. For this algorithm to work properly, the data collection should be in a sorted form.

```
#include <iostream>
using namespace std;
// Recursive Binary Search Function
int binarySearch(const int arr[], int low, int high, int key) {
    if (low > high)          // Base case: element not found
        return -1;
    int mid = low + (high - low) / 2;   // Safe mid calculation
    if (arr[mid] == key)      // If element found
        return mid;
    if (key < arr[mid])       // Search left half
        return binarySearch(arr, low, mid - 1, key);
    return binarySearch(arr, mid + 1, high, key);  // Search right half
}
int main() {
    constexpr int n = 5;
    int arr[n] = {12, 14, 18, 22, 39};
    int key = 22;
```

```
    int result = binarySearch(arr, 0, n - 1, key);
    if (result != -1)
        cout << "Element found at index: " << result << endl;
    else
        cout << "Unsuccessful Search" << endl;
    key = 23;
    result = binarySearch(arr, 0, n - 1, key);
    if (result != -1)
        cout << "Element found at index: " << result << endl;
    else
        cout << "Unsuccessful Search" << endl;
    return 0;
}
```

Output

Element found at index: 3

Unsuccessful Search

CHAPTER 6

SORTING TECHNIQUES

Introduction

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order. The importance of sorting lies in the fact that;

1. Data searching can be optimized to a very high level, if data is stored in a sorted manner.
2. Used to represent data in more readable formats.

Following are some of the examples of sorting in real-life scenarios −

- ✓ **Telephone Directory** − The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- ✓ **Dictionary** − The dictionary stores words in an alphabetical order so that searching of any word becomes easy.

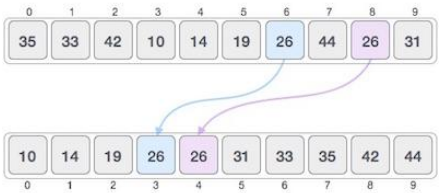
In-place Sorting and Not-in-place Sorting

Sorting algorithms may require some extra space for comparison and temporary storage of few data elements. In-place is where algorithms do not require any extra space. Bubble sort is an example of in-place sorting.

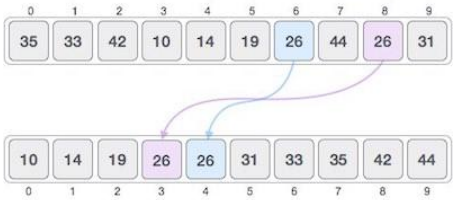
However, in some sorting algorithms, the program requires space which is more than or equal to the elements being sorted. Sorting which uses equal or more space is called **not-in-place sorting**. Merge-sort is an example of not-in-place sorting.

Stable and Not Stable Sorting

If a sorting algorithm, after sorting the contents, does not change the sequence of similar content in which they appear, it is called **stable sorting**.



If a sorting algorithm, after sorting the contents, changes the sequence of similar content in which they appear, it is called **unstable sorting.**



Stability of an algorithm matters when we wish to maintain the sequence of original elements, like in a tuple for example.

Adaptive and Non-Adaptive Sorting Algorithm

A sorting algorithm is said to be **adaptive,** if it takes advantage of already 'sorted' elements in the list that is to be sorted. That is, while sorting if the source list has some element already sorted, adaptive algorithms will take this into account and will try not to re-order them.

A **non-adaptive** algorithm is one which does not take into account the elements which are already sorted. They try to force every single element to be re-ordered to confirm their sorted ness.

Important Terms

Some terms are generally coined while discussing sorting techniques, here is a brief introduction to them −

Increasing Order

A sequence of values is said to be in **increasing order**, if the successive element is greater than the previous one. For example, 1, 3, 4, 6, 8, 9 are in increasing order, as every next element is greater than the previous element.

Decreasing Order

A sequence of values is said to be in **decreasing order**, if the successive element is less than the current one. For example, 9, 8, 6, 4, 3, 1 are in decreasing order, as every next element is less than the previous element.

Non-Increasing Order

A sequence of values is said to be in **non-increasing order**, if the successive element is less than or equal to its previous element in the sequence. This order occurs when the sequence contains duplicate values. For example, 9, 8, 6, 3, 3, 1 are in non-increasing order, as every next element is less than or equal to (in case of 3) but not greater than any previous element.

Non-Decreasing Order

A sequence of values is said to be in **non-decreasing order**, if the successive element is greater than or equal to its previous element in the sequence. This order occurs when the sequence contains

duplicate values. For example, 1, 3, 3, 6, 8, 9 are in non-decreasing order, as every next element is greater than or equal to (in case of 3) but not less than the previous one.

There are several sorting techniques available to sort the contents of various data structures. Following are some of those −

- ✓ Bubble Sort
- ✓ Insertion Sort
- ✓ Selection Sort
- ✓ Merge Sort
- ✓ Shell Sort
- ✓ Heap Sort
- ✓ Quick Sort

Bubble Sort Algorithm

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of O(n2) where n is the number of items.

Bubble Sort Algorithm

Bubble Sort is an elementary sorting algorithm, which works by repeatedly exchanging adjacent elements, if necessary. When no exchanges are required, the file is sorted.

We assume list is an array of n elements. We further assume that swap function swaps the values of the given array elements.

Step 1 − Check if the first element in the input array is greater than the next element in the array.

Step 2 − If it is greater, swap the two elements; otherwise move the pointer forward in the array.

Step 3 − Repeat Step 2 until we reach the end of the array.

Step 4 − Check if the elements are sorted; if not, repeat the same process (Step 1 to Step 3) from the last element of the array to the first.

Step 5 − The final output achieved is the sorted array.

Algorithm: Sequential-Bubble-Sort (A)

For i ← 1 to length [A] do

for j ← length [A] down-to i +1 do

if A[A] < A[j-1] then

Exchange A[j] ↔ A[j-1]

Pseudocode

We observe in algorithm that Bubble Sort compares each pair of array element unless the whole array is completely sorted in an ascending order. This may cause a few complexity issues like what if the array needs no more swapping as all the elements are already ascending.

To ease-out the issue, we use one flag variable swapped which will help us see if any swap has happened or not. If no swap has occurred, i.e. the array requires no more processing to be sorted; it will come out of the loop.

Pseudocode of bubble sort algorithm can be written as follows −

```
Void bubbleSort(int numbers[], int array_size){
  inti, j, temp;
  for (i = (array_size - 1); i>= 0; i--)
  for (j = 1; j <= i; j++)
  if (numbers[j-1] > numbers[j]){
    temp = numbers[j-1];
    numbers[j-1] = numbers[j];
    numbers[j] = temp;
  }
}
```

Analysis

Here, the number of comparisons are

$$1 + 2 + 3 + ... + (n - 1) = n(n - 1)/2 = O(n^2)$$

Clearly, the graph shows the n^2 nature of the bubble sort.

In this algorithm, the number of comparison is irrespective of the data set, i.e. whether the provided input elements are in sorted order or in reverse order or at random.

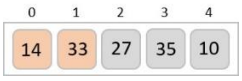
Memory Requirement

From the algorithm stated above, it is clear that bubble sort does not require extra memory.

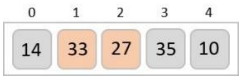
Example

We take an unsorted array for our example. Bubble sort takes (n^2) time so we're keeping it short and precise.

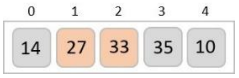
Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



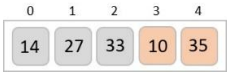
We find that 27 is smaller than 33 and these two values must be swapped.



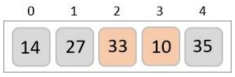
Next we compare 33 and 35. We find that both are in already sorted positions.

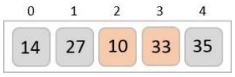
Then we move to the next two values, 35 and 10.

We know then that 10 is smaller 35. Hence, they are not sorted. We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this −

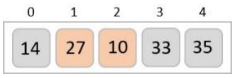


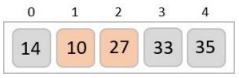
To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this −

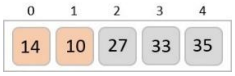


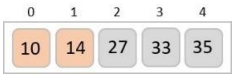


Notice that after each iteration, at least one value moves at the end.

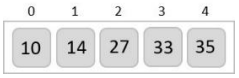








In addition, when there is no swap required, bubble sort learns that an array is completely sorted.



Now we should look into some practical aspects of bubble sort.

Implementation

One more issue we did not address in our original algorithm and its improvised pseudocode, is that, after every iteration the highest values settles down at the end of the array. Hence, the next iteration need not include already sorted elements. For this purpose, in our implementation, we restrict the inner loop to avoid already sorted values.

```
#include<iostream>
using namespace std;
//Function to perform BubbleSort on an array
//array[] receives the array to be sorted.
//size represents the number of elements in the array.
void bubbleSort(int array[], int size){
   //Outer loop controls the number of passes
  for(int i = 0; i<size; i++) {
     bool swaps = false; // A flag variable used to check if any swapping occurs during a pass. If no swaps occur, the array is already sorted
     //Inner loop compares an adjacent elements
     for(int j = 0; j<size-i-1; j++) {
         // Check if the current element is greater than the next element. size - i - 1 reduces comparisons because the last elements are already sorted after each pass
       if(array[j] > array[j+1]) { //when the current item is bigger than next.
         swap(array[j], array[j + 1]); // Swap the two elements
         swaps = true; //set swap flag
       }
     }
```

```
    if(!swaps)
      break; // No swap in this pass, so array is sorted
  }
}
int main(){
  int n; //stores the number of elements.
  n = 5;
  int arr[5] = {67, 44, 82, 17, 20}; //initialize an array
  cout << "Array before Sorting: ";
  for(int i = 0; i<n; i++)
    cout << arr[i] << " ";
  cout << endl;
  bubbleSort(arr, n); //alls the bubbleSort function to sort the array.
  cout << "Array after Sorting: ";
  for(int i = 0; i<n; i++)
    cout << arr[i] << " ";
  cout << endl;
}
```

Output

Array before Sorting: 67 44 82 17 20

Array after Sorting: 17 20 44 67 82

Insertion Sort Algorithm

Insertion sort is a very simple method to sort numbers in an ascending or descending order. This method follows the incremental method. It can be compared with the technique how cards are sorted at the time of playing a game.

This is an **in-place comparison-based** sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element, which is to be 'inserted' in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence, the name, insertion sort.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst-case complexity are of (n^2), where n is the number of items.

Insertion Sort Algorithm

Now we have a bigger picture of how this sorting technique works, so we can derive simple steps by which we can achieve insertion sort.

Step 1− If it is the first element, it is already sorted. return 1;

Step 2− Pick next element

Step 3− Compare with all elements in the sorted sub-list

Step 4− Shift all the elements in the sorted sub-list that is greater than the value to be sorted

Step 5− Insert the value

Step 6− Repeat until list is sorted

Pseudocode

```
Algorithm: Insertion-Sort(A)
for j = 2 to A.length
  key = A[j]
  i = j  1
  while i > 0 and A[i] > key
    A[i + 1] = A[i]
    i = i -1
  A[i + 1] = key
```

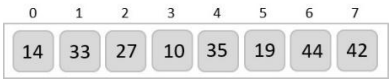
Analysis

Run time of this algorithm is very much dependent on the given input.

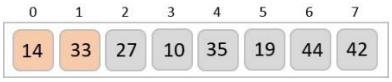
If the given numbers are sorted, this algorithm runs in O(n) time. If the given numbers are in reverse order, the algorithm runs in O(n^2) time.

Example

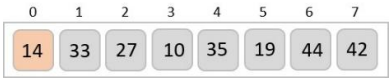
We take an unsorted array for our example.



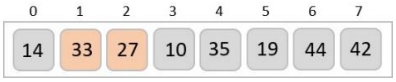
Insertion sort compares the first two elements.



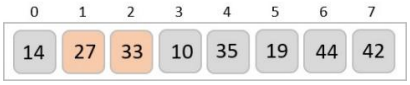
It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sub-list.



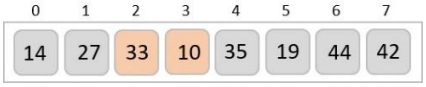
Insertion sort moves ahead and compares 33 with 27.



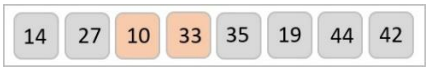
In addition, finds that 33 is not in the correct position. It swaps 33 with 27. It also checks with all the elements of sorted sub-list. Here we see that the sorted sub-list has only one element 14 and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.



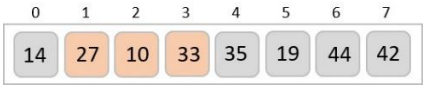
By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10. These values are not in a sorted order.



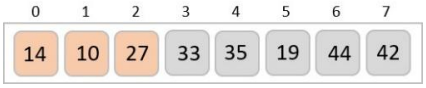
So they are swapped.



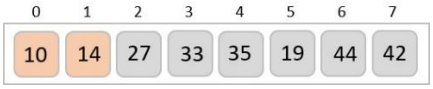
However, swapping makes 27 and 10 unsorted.



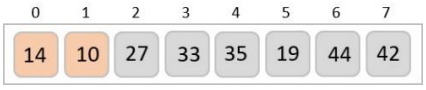
Hence, we swap them too.



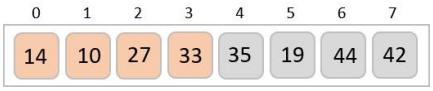
Again we find 14 and 10 in an unsorted order.



We swap them again.



By the end of third iteration, we have a sorted sub-list of 4 items.



Implementation

Since insertion sort is an in-place sorting algorithm, the algorithm is implemented in a way where the key element which is iteratively chosen as every element in the array is compared with it consequent elements to check its position. If the key element is less than its successive element, the swapping is not done. Otherwise, the two elements compared will be swapped and the next element is chosen as the key element.

```
#include<iostream> // Includes input-output stream library
using namespace std; // Allows use of standard C++ names without std::
// Function to perform Insertion Sort
void insertionSort(int arr[], int size){
  int key, pos; // key stores current element, pos is used for shifting
  // Loop starts from index 1 because first element is assumed sorted
  for(int i = 1; i < size; i++) {
    key = arr[i]; // Store the current element to be inserted
    pos = i; // Set position to current index
    // Shift elements that are greater than key to the right
    while(pos > 0 && arr[pos - 1] > key) {
      arr[pos] = arr[pos - 1]; // Move larger element one position ahead
      pos--; // Move to the previous index
    }
    // Insert the key at its correct position
    arr[pos] = key;
  }
}
int main(){
  int n; // Variable to store number of elements
  n = 5; // Assign size of the array
  int arr[5] = {67, 44, 82, 17, 20}; // Declare and initialize the array
  // Display array before sorting
```

```
  cout << "Array before Sorting: ";
  for(int i = 0; i < n; i++)
    cout << arr[i] << " ";
  cout << endl; // Move to a new line
  // Call insertion sort function
  insertionSort(arr, n);
  // Display array after sorting
  cout << "Array after Sorting: ";
  for(int i = 0; i < n; i++)
    cout << arr[i] << " ";
  cout << endl; // Move to a new line
}
```

Output

Array before Sorting: 67 44 82 17 20

Array after Sorting: 17 20 44 67 82

Selection Sort Algorithm

Selection sort is a simple sorting algorithm. This sorting algorithm, like insertion sort, is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundaries by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of **$O(n^2)$**, where **n** is the number of items.

Selection Sort Algorithm

This type of sorting is called Selection Sort as it works by repeatedly sorting elements. That is: we first find the smallest value in the array and exchange it with the element in the first position, then find the second smallest element and exchange it with the element in the second position, and we continue the process in this way until the entire array is sorted.

1. Set MIN to location 0.

2. Search the minimum element in the list.
3. Swap with value at location MIN.
4. Increment MIN to point to next element.
5. Repeat until the list is sorted.

Pseudocode

```
Algorithm: Selection-Sort (A)
fori← 1 to n-1 do
  min j ←i;
  min x ← A[i]
  for j ←i + 1 to n do
    if A[j] < min x then
      min j ← j
      min x ← A[j]
  A[min j] ← A [i]
  A[i] ← min x
```

Analysis

Selection sort is among the simplest of sorting techniques and it works very well for small files. It has a quite important application as each item is actually moved at the most once.

Section sort is a method of choice for sorting files with very large objects (records) and small keys.

The worst case occurs if the array is already sorted in a descending order and we want to sort them in an ascending order.

Nonetheless, the time required by selection sort algorithm is not very sensitive to the original order of the array to be sorted: the test if [] < ***A[j] < min x*** is executed exactly the same number of times in every case.

Selection sort spends most of its time trying to find the minimum element in the unsorted part of the array. It clearly shows the similarity between Selection sort and Bubble sort.

Bubble sort selects the maximum remaining elements at each stage, but wastes some effort imparting some order to an unsorted part of the array.

Selection sort is quadratic in both the worst and the average case, and requires no extra memory.

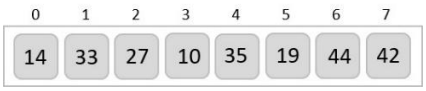
For each **i** from 1 to n - 1, there is one exchange and n - i comparisons, so there is a total of n - 1 exchanges and **(n 1) + (n 2) + ...+2 + 1 = n(n 1)/2** comparisons.

These observations hold, no matter what the input data is.

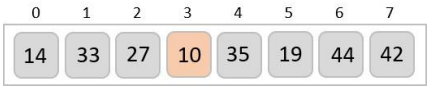
In the worst case, this could be quadratic, but in the average case, this quantity is **O(n log n)**. It implies that the **running time of Selection sort is quite insensitive to the input.**

Example

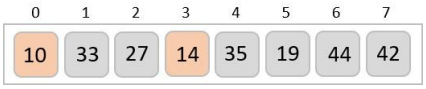
Consider the following depicted array as an example.



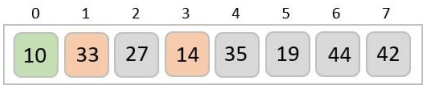
For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



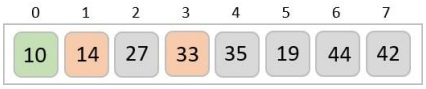
So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



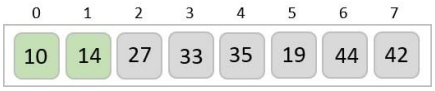
For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



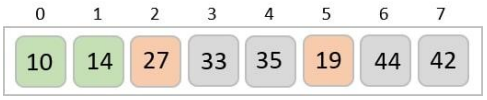
We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

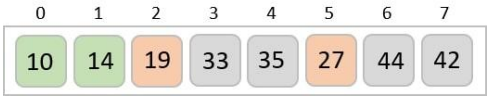


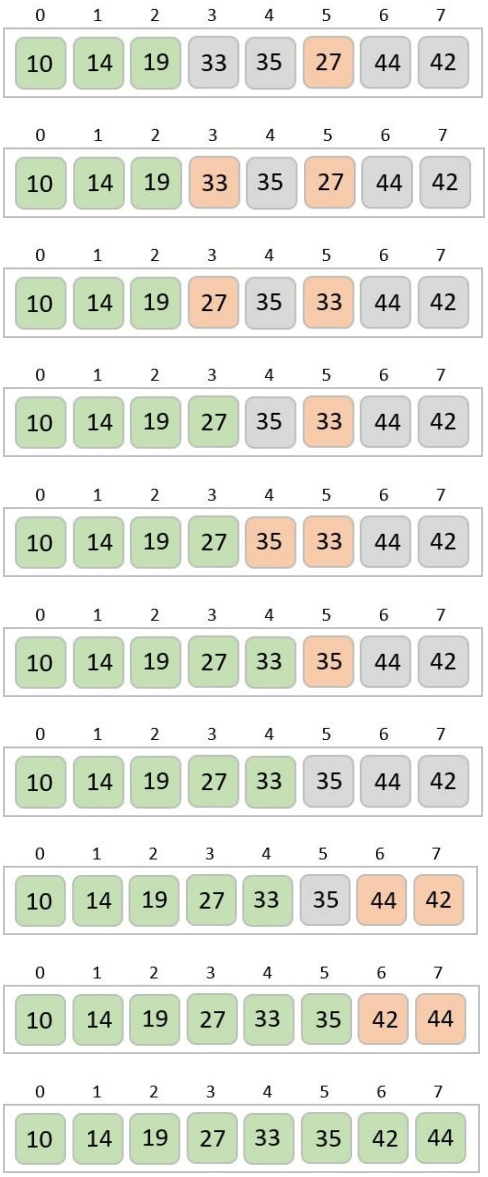
After two iterations, two least values are positioned at the beginning in a sorted manner.



The same process is applied to the rest of the items in the array







Implementation

The selection sort algorithm is implemented in four different programming languages below. The given program selects the minimum number of the array and swaps it with the element in the first index. The second minimum number is swapped with the element present in the second index. The process goes on until the end of the array is reached.

```
#include <iostream>        // Allows input and output operations
using namespace std;       // Allows use of standard names like cout without std::
// Function that sorts an array using Selection Sort
void selectionSort(int arr[], int n) {
```

```
    // Loop through the array elements one by one
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;   // Assume the current position has the smallest value
        // Loop to find the smallest element in the remaining unsorted part
        for (int j = i + 1; j < n; j++) {
                    // Compare current element with the smallest found so far
            if (arr[j] < arr[minIndex]) {
                minIndex = j;  // Update index of the smallest element
            }
        }
          // Swap the smallest element with the element at position i
        int temp = arr[i];        // Store current element in a temporary variable
        arr[i] = arr[minIndex];   // Put the smallest element at position i
        arr[minIndex] = temp;     // Put the original element in the minIndex position
    }
}
int main() {
    // Declare and initialize an integer array
    int arr[] = {12, 19, 55, 2, 16};
    // Store the number of elements in the array
    int n = 5;
    // Display array elements before sorting
    cout << "Before sorting: ";
    for (int i = 0; i < n; i++)     // Loop through the array
        cout << arr[i] << " ";     // Print each element
    // Call the selection sort function
    selectionSort(arr, n)
    // Display array elements after sorting
    cout << "\nAfter sorting: ";
    for (int i = 0; i < n; i++)     // Loop through the sorted array
        cout << arr[i] << " ";     // Print each sorted element
```

```
    return 0;   // End of the program
}
```

Output

Array before Sorting: 12 19 55 2 16

Array after Sorting: 2 12 16 19 55

Merge Sort Algorithm

Merge sort is a sorting technique based on divide and conquer technique. With worst-case time complexity being (n log n), it is one of the most used and approached algorithms.

Merge sort first divides the array into equal halves and then combines them in a sorted manner.

How Merge Sort Works?

To understand merge sort, we take an unsorted array as the following −

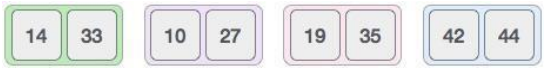
We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.

This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.

We further divide these arrays and we achieve atomic value which can no more be divided.

Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.

After the final merging, the list becomes sorted and is considered the final solution.

Merge Sort Algorithm

Merge sort keeps on dividing the list into equal halves until it can no more be divided. By definition, if it is only one element in the list, it is considered sorted. Then, merge sort combines the smaller sorted lists keeping the new list sorted too.

1. Step 1: If it is only one element in the list, consider it already sorted, so return.
2. Step 2: Divide the list recursively into two halves until it can no more be divided.
3. Step 3: Merge the smaller lists into new list in sorted order.

Pseudocode

We shall now see the pseudocodes for merge sort functions. As our algorithms point out two main functions divide & merge.

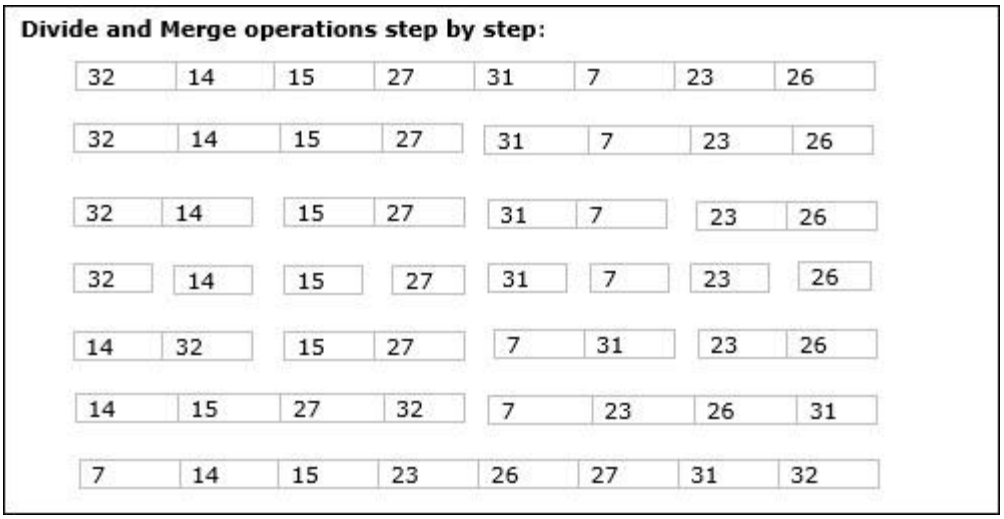
Merge sort works with recursion and we shall see our implementation in the same way.

```
procedure mergesort( var a as array )
   if ( n == 1 ) return a
      var l1 as array = a[0] ... a[n/2]
      var l2 as array = a[n/2+1] ... a[n]
      l1 = mergesort( l1 )
      l2 = mergesort( l2 )
      return merge( l1, l2 )
end procedure
procedure merge( var a as array, var b as array )
   var c as array
   while ( a and b have elements )
      if ( a[0] > b[0] )
         add b[0] to the end of c
         remove b[0] from b
```

```
    else
      add a[0] to the end of c
      remove a[0] from a
    end if
  end while
  while ( a has elements )
    add a[0] to the end of c
    remove a[0] from a
  end while
  while ( b has elements )
    add b[0] to the end of c
    remove b[0] from b
  end while
  return c
end procedure
```

Example

In the following example, we have shown Merge-Sort algorithm step by step. First, every iteration array is divided into two sub-arrays, until the sub-array contains only one element. When these sub-arrays cannot be divided further, then merge operations are performed.



Analysis

Let us consider, the running time of Merge-Sort as $T(n)$. Hence,

T(n)={c2xT(n2)+dxnifn≤1otherwisewherecanddareconstants

Therefore, using this recurrence relation,

$T(n)=2iT(n/2i)+i\cdot d\cdot n$

As,$i=\log n$,$T(n)=2\log nT(n/2\log n)+\log n\cdot d\cdot n$

$=c\cdot n+d\cdot n\cdot \log n$

Therefore,$T(n)=O(n\log n)$.

Implementation

```
#include <iostream>   // Includes input and output stream library
using namespace std;  // Allows use of standard names like cout and cin without std::
const int MAX = 10;   // Declares a constant integer MAX with value 10 (maximum array size)
// Function to merge two sorted subarrays
// arr[]  -> original array
// temp[] -> temporary array used for merging
// low    -> starting index of first subarray
// mid    -> ending index of first subarray
// high   -> ending index of second subarray
void merge(int arr[], int temp[], int low, int mid, int high) {
    int i = low;        // i points to the beginning of the left subarray
    int j = mid + 1;    // j points to the beginning of the right subarray
    int k = low;        // k points to the position in temp[] where merged elements go
    // Compare elements from both subarrays and store smaller one in temp[]
    while (i <= mid && j <= high) {   // Continue while both subarrays have elements
        if (arr[i] <= arr[j])         // If left element is smaller or equal
            temp[k++] = arr[i++];     // Copy left element to temp and move i and k forward
        else
            temp[k++] = arr[j++];     // Otherwise copy right element and move j and k forward
    }
    // Copy remaining elements of left subarray (if any remain)
    while (i <= mid)
        temp[k++] = arr[i++];         // Copy remaining left elements into temp

    // Copy remaining elements of right subarray (if any remain)
```

```
    while (j <= high)
        temp[k++] = arr[j++];         // Copy remaining right elements into temp
    // Copy merged elements from temp[] back into original array arr[]
    for (int x = low; x <= high; x++)
        arr[x] = temp[x];             // Overwrite original array with sorted elements
}
// Recursive Merge Sort function
// This function divides the array into smaller parts until each part has one element
void mergeSort(int arr[], int temp[], int low, int high) {
    if (low < high) {                 // If more than one element exists
        int mid = (low + high) / 2;       // Find middle index
        mergeSort(arr, temp, low, mid);        // Recursively sort left half
        mergeSort(arr, temp, mid + 1, high);   // Recursively sort right half
        merge(arr, temp, low, mid, high);      // Merge the two sorted halves
    }
}
// Main function (program execution starts here)
int main() {
    int arr[MAX];    // Declare array to store user input
    int temp[MAX];   // Temporary array used during merging
    cout << "Enter 10 elements:\n";   // Prompt user to enter elements
    // Loop to read 10 elements from the user
    for (int i = 0; i < MAX; i++) {
        cin >> arr[i];                // Store input into array
    }
    cout << "\nArray before sorting:\n";  // Display message
    // Loop to display array before sorting
    for (int i = 0; i < MAX; i++) {
        cout << arr[i] << " ";        // Print each element
    }
    // Call mergeSort function
```

```
    // 0 is starting index
    // MAX - 1 is last index (9)
    mergeSort(arr, temp, 0, MAX - 1);
    cout << "\n\nArray after sorting:\n";  // Display message
    // Loop to display sorted array
    for (int i = 0; i < MAX; i++) {
        cout << arr[i] << " ";        // Print sorted elements
    }
    return 0;   // Indicates successful program termination
}
```

Output

Array before sorting

10 14 19 26 27 31 33 35 42 44 0

Array after sorting

0 10 14 19 26 27 31 33 35 42 44

Quick Sort

Quick sort is a highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than the specified value, say pivot, based on which the partition is made and another array holds values greater than the pivot value.

Quicksort partitions an array and then calls itself recursively twice to sort the two resulting subarrays. This algorithm is quite efficient for large-sized data sets as its average and worst-case complexity are O(n2), respectively.

Quick Sort Pivot Algorithm

Based on our understanding of partitioning in quick sort, we will now try to write an algorithm for it, which is as follows.

1. Choose the highest index value has pivot
2. Take two variables to point left and right of the list
excluding pivot
3. Left points to the low index
4. Right points to the high

5. While value at left is less than pivot move right

6. While value at right is greater than pivot move left

7. If both step 5 and step 6 does not match swap left and right

8. If left ≥ right, the point where they met is new pivot

Quick Sort Pivot Pseudocode

The pseudocode for the above algorithm can be derived as −

```
function partitionFunc(left, right, pivot)
  leftPointer = left
  rightPointer = right - 1
  while True do
    while A[++leftPointer] < pivot do
    //do-nothing
    end while
    while rightPointer > 0 && A[--rightPointer] > pivot do
      //do-nothing
    end while
    if leftPointer >= rightPointer
      break
    else
      swap leftPointer,rightPointer
    end if
  end while
  swap leftPointer,right
  return leftPointer
end function
```

Quick Sort Algorithm

Using pivot algorithm recursively, we end up with smaller possible partitions. Each partition is then processed for quick sort. We define recursive algorithm for quicksort as follows −

1. Make the right-most index value pivot

2. Partition the array using pivot value

3. Quicksort left partition recursively

4. Quicksort right partition recursively

Quick Sort Pseudocode

To get more into it, let see the pseudocode for quick sort algorithm −

```
procedure quickSort(left, right)
   if right-left <= 0
      return
   else
      pivot = A[right]
      partition = partitionFunc(left, right, pivot)
      quickSort(left,partition-1)
      quickSort(partition+1,right)
   end if
end procedure
```

Analysis

The worst case complexity of Quick-Sort algorithm is O(n2). However, using this technique, in average cases generally we get the output in O (n log n) time.

Implementation

```
#include <iostream>          // Allows use of input/output functions
using namespace std;         // Avoids writing std:: before cout, cin, etc.
// Declare a constant for array size (modern replacement for #define)
const int MAX = 7;

// Function to display array elements
void display(int arr[], int size) {
    cout << "[";                    // Print opening bracket
    // Loop through the array
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";       // Print each element followed by space
    }
    cout << "]\n";                  // Print closing bracket and move to next line
}
```

```
// Function to swap two elements in the array
void swap(int arr[], int index1, int index2) {
    int temp = arr[index1];          // Store first value temporarily
    arr[index1] = arr[index2];       // Replace first value with second
    arr[index2] = temp;              // Replace second with stored value
}
// Partition function
// Rearranges elements around a pivot
int partition(int arr[], int left, int right) {
    int pivot = arr[right];          // Choose last element as pivot
    int i = left - 1;                // Pointer for smaller element
    // Loop through elements from left to right-1
    for (int j = left; j < right; j++) {
        // If current element is smaller than pivot
        if (arr[j] <= pivot) {
            i++;                     // Move boundary of smaller elements
            swap(arr, i, j);         // Swap current element with boundary
        }
    }
    // Place pivot in its correct sorted position
    swap(arr, i + 1, right);
    return i + 1;                    // Return pivot index
}
// QuickSort function (Recursive)
void quickSort(int arr[], int left, int right) {
    // Base condition: if left index is less than right
    if (left < right) {
        // Partition the array and get pivot index
        int pivotIndex = partition(arr, left, right);
        // Recursively sort elements before pivot
        quickSort(arr, left, pivotIndex - 1);
```

```
        // Recursively sort elements after pivot
        quickSort(arr, pivotIndex + 1, right);
    }
}
// Main function
int main() {
    // Initialize array with given values
    int intArray[MAX] = {4, 6, 3, 2, 1, 9, 7};
    cout << "Input Array: ";
    display(intArray, MAX);          // Display original array
    // Call quickSort
    quickSort(intArray, 0, MAX - 1);
    cout << "Output Array: ";
    display(intArray, MAX);          // Display sorted array
    return 0;                        // End program successfully
}
```